

Entwicklung einer Capability-basierten Zugriffskontrolle in einem Rust-Betriebssystem

Masterarbeit

vorgelegt von

Julian Linus Schnock

02. April 2026

im Studiengang Informatik
zur Erlangung des akademischen Grades

Master of Science (M.Sc.)

Erstgutachter: Univ.-Prof. Dr. M. Schöttner
Zweitgutachter: Univ.-Prof. Dr. M. Mauve

Selbstständigkeitserklärung

Hiermit versichere ich, die vorliegende Masterarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben. Alle Stellen, die aus den Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Düsseldorf, den 02. April 2026

Julian Linus Schnock

Zusammenfassung

Die vorliegende Arbeit untersucht und implementiert eine Capability-basierte Zugriffskontrolle für das Forschungsbetriebssystem D3OS, das als verteiltes, in Rust entwickeltes Mikrokern-System für moderne Rechenzentrumsanforderungen konzipiert ist.

Die Motivation der Arbeit liegt darin, ein flexibleres und sichereres Zugriffskontrollmodell zu entwickeln, das den Anforderungen moderner, verteilter Systeme gerecht wird. Capability-basierte Zugriffskontrolle verfolgt dabei einen dezentralen Ansatz: Zugriffsrechte werden nicht global verwaltet, sondern als nicht manipulierbare Objekte direkt an Prozesse vergeben. Diese Capabilities definieren präzise, auf welche Ressourcen und in welcher Weise zugegriffen werden darf. Dadurch wird das Prinzip der minimalen Rechtevergabe konsequent umgesetzt und die Angriffsfläche reduziert.

Im theoretischen Teil werden Grundlagen der Zugriffskontrolle sowie bestehende Systeme wie seL4, NOVA oder Fuchsia analysiert. Diese dienen als Referenz für Designentscheidungen, insbesondere hinsichtlich Capability-Spaces, Rechteweitergabe und Systemintegration. Zudem werden zentrale Herausforderungen wie Revocation, also der Entzug von Rechten, und sichere Delegation behandelt.

Der praktische Schwerpunkt der Arbeit liegt auf der Implementierung des Capability-Modells in D3OS. Hierzu wurden zentrale Systemkomponenten angepasst. Capabilities werden als generische Datenstrukturen implementiert, die sowohl das geschützte Objekt als auch zugehörige Zugriffsrechte kapseln. Alle Capabilities eines Prozesses werden in einem Capability-Space verwaltet, der im Kernel liegt, um Manipulationen zu verhindern. Prozesse und Threads greifen ausschließlich über diese Capabilities auf Ressourcen zu.

Ein wesentlicher Beitrag ist die Umstellung der Systemaufrufe: Statt über globale Syscall-Nummern werden Aufrufe nun über Capability-Referenzen gesteuert. Dadurch kann ein Prozess nur noch solche Operationen ausführen, für die er explizit berechtigt ist. Auch das Dateisystem und das Naming-System wurden entsprechend angepasst, sodass Zugriffe nur noch über vorhandene Capabilities möglich sind. Klassische globale Sichtweisen auf Ressourcen werden dadurch eingeschränkt. Zusätzlich wurden Mechanismen zur Weitergabe und zum Entzug von Capabilities implementiert.

Die Ergebnisse zeigen, dass sich eine Capability-basierte Zugriffskontrolle effektiv in ein modernes Betriebssystem integrieren lässt und eine feingranulare sowie sichere Rechteverwaltung ermöglicht. Gleichzeitig ergeben sich neue Herausforderungen, etwa bei der sicheren Kommunikation zwischen Prozessen oder der effizienten Verwaltung von Revocations. Insgesamt bildet die Arbeit eine solide Grundlage für weiterführende Entwicklungen, insbesondere im Hinblick auf Restartability, Message-Passing und verteilte Systeme.

Inhaltsverzeichnis

Abbildungsverzeichnis	x
Quellcodeverzeichnis	x
1 Einleitung	1
1.1 D3OS	1
1.2 Mikrokernel	2
1.3 Prinzipien von Zugriffskontrollen	2
2 Zentral-organisierte Zugriffskontrollen	4
2.1 Role-based Access-Control (RBAC)	4
2.2 Attribute-based Access-Control (ABAC))	5
3 Capability-basierte Zugriffskontrolle	6
3.1 Was ist eine Capability-based Access-Control (CapBAC)?	6
3.1.1 Revocation	7
3.2 Historie und Entwicklung des Modells	8
3.2.1 Das Confused-Deputy Problem	8
3.2.2 Limitierungen unter ACL	9
3.2.3 Die Lösung	9
4 Existierende Systeme	11
4.1 L4	11
4.2 seL4	11
4.3 NOVA	11
4.4 Mungi	12
4.5 J-Kernel	12
4.6 Capsicum	13
4.7 Google Fuchsia	14
4.8 EROS	14
4.9 Relevanz der betrachteten Systeme für D3OS	15
5 Implementierung	17

5.1	Capability	17
5.2	Capability-Space	18
5.2.1	Kernel-Integration	19
5.2.2	User-API	22
5.3	Syscall-Management	23
5.4	Naming-System	23
5.4.1	Bisherige Implementierung	23
5.4.2	Capability Support	24
5.5	Share-Mechanismus	25
5.6	Revocation	28
5.6.1	Selektiv und Partiell	29
5.6.2	Transitiv	30
5.6.3	Independent	30
5.6.4	Immediate	31
5.6.5	Temporal	31
6	Capabilities im Mikrokernel	33
6.1	Theorie	33
6.2	Treiber-interne Rechteverwaltung	34
6.3	Demonstration	34
6.3.1	Treiber	34
6.3.2	Nutzer	37
7	Zusammenfassung und Ausblick	41
7.1	Messaging	42
7.2	Multicore	43
7.3	Restartability	43
7.4	Shared-Memory	44
7.5	Zusammenfassung	45
Anhang A	Quellcode	46
Anhang B	Unterstützende Nutzung generativer KI-Werkzeuge	46
Anhang C	Benchmarks	46

Abbildungsverzeichnis

1	D3OS Logo[Bet24a]	1
2	Aufbau Mikrokern und Monolith [IOA ⁺ 21a]	2
3	Role-Based Access-Control	4
4	Confused Deputy Problem	8
5	Capability Spaces in Nova [Ste13]	12
6	Java-Objektreferenzen mit Zugriffsrechten kombinieren[HCC ⁺ 97]	13
7	Capability-Modus zu Unix-Dateideskriptoren hinzufügen[WALK10]	13
8	Vererbungshierarchie[Goo25]	14
9	CSpace Konfiguration	20
10	Demo einer geteilten Pipe	27
11	Demo einer entzogenen Naming Capability	28
12	Demo des Dateitreibers	40
13	Channel-Architektur in Google-Fuchsia[Goo25]	42
14	Speichersegment Capability [HRK91]	44

Quellcodeverzeichnis

1	Capability Struct (capability.rs)	17
2	CSpace Struct (cspace.rs)	18
3	Prozess (process.rs)	19
4	Thread (thread.rs)	19
5	Initialize CSpace (cspace.rs)	21
6	User Capability (/library/capabilities/src/capability.rs)	22
7	Pipe Erstellen durch User	24
8	Share (capability.rs)	25
9	Share Test-Anwendung (pipetest.rs)	26
10	Testanwendung (revoketest.rs & revoketest1.rs)	29
11	Start des Dateitreibers (filesrvr.rs)	34
12	Warten bis eine Capability geteilt wird (filesrvr.rs)	35
13	Anfrageverarbeitung (filesrvr.rs)	36
14	Einstieg der Nutzeranwendung (fileclient.rs)	37
15	Aufbauen der Verbindung (fileclient.rs)	38
16	Befehle an den Dateitreiber (fileclient.rs)	39

1 Einleitung

Im Laufe dieser Arbeit wird eine Capability-basierte Zugriffskontrolle für das Forschungsbetriebssystem „D3OS“ des Lehrstuhls für Betriebssysteme an der Heinrich-Heine Universität Düsseldorf implementiert.

Bevor auf die Details der Implementierung sowie die zugrunde liegenden theoretischen Konzepte eingegangen wird, erfolgt zunächst eine Einführung in das Zielsystem. Anschließend werden grundlegende Prinzipien von Zugriffskontrollen erläutert, die als Basis für die spätere Implementierung dienen.

1.1 D3OS



Abbildung 1: D3OS Logo[\[Bet24a\]](#)

D3OS (Distributed Data-Driven Operating System) ist ein Betriebssystem, das am Lehrstuhl für Betriebssysteme der Heinrich-Heine-Universität Düsseldorf in der Programmiersprache Rust entwickelt wird [\[Bet24b\]](#).

Das System wird speziell für moderne Rechenzentrumshardware konzipiert und soll skalierbares Big-Data-Computing ermöglichen. D3OS strebt einen verteilten Mikro-Kernel-Ansatz an. Es behandelt die zugrunde liegende Hardware als verteiltes System und will dabei selbst verteilt agieren.

Darüber hinaus soll die herkömmliche Trennung zwischen Dateien und im Speicher gehaltenen Daten aufgehoben werden. Das Forschungsteam sieht ein solches Betriebssystem als notwendig an, da bestehende Systeme ihrer Einschätzung nach nicht ausreichend auf die Anforderungen von Big-Data-Anwendungen und neuen Rechenzentrumsarchitekturen ausgelegt sind.

1.2 Mikrokernel

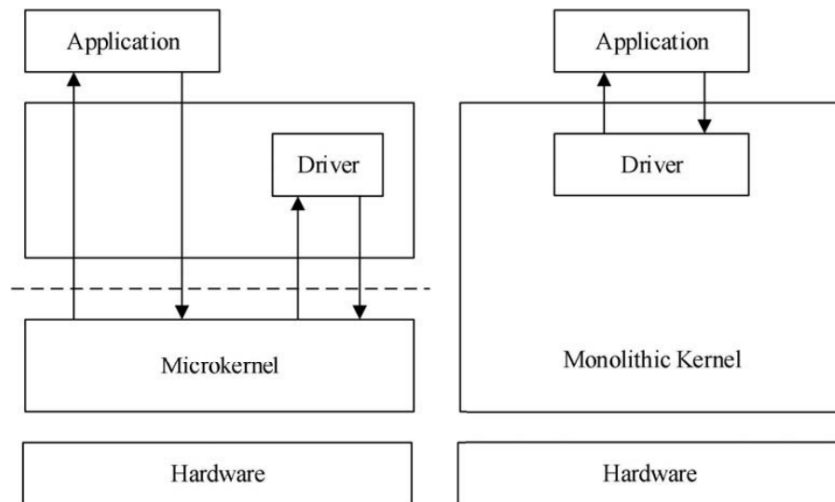


Abbildung 2: Aufbau Mikrokernel und Monolith [IOA⁺21a]

Im Gegensatz zu monolithischen Betriebssystemen implementieren Mikrokernel lediglich minimale Kernfunktionen wie Scheduling, Interprozesskommunikation (IPC) und grundlegende Speicherverwaltung. Alle weiteren Dienste, insbesondere Treiber, Dateisysteme und Netzwerkdienste, werden in den Userspace ausgelagert und über definierte IPC-Schnittstellen angebunden. Dadurch wird eine stärkere Isolation zwischen Systemkomponenten erreicht, da Fehler in einem Userspace-Dienst in der Regel nicht das Gesamtsystem kompromittieren. Es entstehen allerdings zusätzliche Kommunikationskosten durch die verstärkte Nutzung von IPC [Lie95].

Es existieren Hybridformen, bei denen vereinzelte Treiber im Kernel liegen. Ein typisches Beispiel sind Dateitreiber, da Platten-Zugriffe zur Gewährung der Zugriffskontrolle über den Kernel durchgeführt werden sollten [Wik25b].

1.3 Prinzipien von Zugriffskontrollen

Um eine technisch solide Zugriffskontrolle zu entwickeln, ist es zunächst notwendig, deren grundlegende Voraussetzungen und Prinzipien zu erläutern:

Das Kontrollieren von Zugriffen wie etwa auf den Arbeitsspeicher, Systemressourcen/-objekten und vielem mehr, ist neben Informationsflusskontrolle und dem Überprüfen von Zertifikaten einer von drei großen Bestandteilen der Betriebssystem-Sicherheit [Sta12].

Um die Sicherheit der verschiedenen Systemressourcen zu gewährleisten, müssen vier wichtige Kriterien erfüllt sein [Sta12]:

1. Informationen dürfen nur lesbar sein, wenn das lesende Programm die notwendigen Berechtigungen besitzt. Hierzu zählt auch die simple Auskunft über die Existenz eines Objektes (Confidentiality).
2. Änderungen dürfen nur mit den notwendigen Rechten stattfinden, dazu zählt auch das Löschen von Daten (Integrity).
3. Systemressourcen müssen für Zugriffe zur Verfügung stehen (wenn die Berechtigungen für diese vorliegen) (Availability).
4. Nutzer müssen verifiziert und identifiziert werden und somit Berechtigungen erteilt werden können (Authentication).

Zur Erleichterung der Erfüllung dieser Merkmale wird empfohlen, neun Designkriterien einzuhalten [SS75]:

- Economy of Mechanism: Sämtliche Systemkomponenten sollten so klein und simpel wie möglich gehalten werden
- Fail-Safe defaults: Sicherheitsmechanismen sollten Regeln besitzen, unter welchen Bedingungen ein Zugriff gestattet wird, nicht wann er verboten wird. Der Standard sollte sein, keine Berechtigung zu haben.
- Complete Mediation: Jeder Zugriff auf jede Ressource muss überprüft werden.
- Open Design: Die Sicherheit sollte nicht darauf basieren, dass ein Angreifer nicht weiß, wie sie funktioniert. Das Design sollte offen zugänglich sein und dafür die Sicherheit auf Passwörter und Sicherheitsobjekte (Keys) beruhen.
- Separation of Privilege: Ein Zugriff, der zwei Arten der Autorisierung verlangt (etwa zwei Schlüssel) ist sicherer als einer, der nur eine benötigt.
- Least Privilege: Jedes Programm und jeder Nutzer sollte nur die minimalen Berechtigungen besitzen, die benötigt werden, um die jeweiligen Aufgaben zu erfüllen.
- Least-Common Mechanism: Minimiere die Anzahl der Nutzer/Programme, die zeitgleich auf einen Mechanismus zugreifen können. Jeder geteilte Zugriff ist ein Weg, um Informationen auszutauschen und zu erhalten.
- Psychological Acceptability: Das System sollte auf eine einfache Benutzung ausgelegt sein, um zu gewährleisten, dass Nutzer die Sicherheitsmechanismen korrekt verwenden.

2 Zentral-organisierte Zugriffskontrollen

Vor der Verbreitung Capability-basierter Zugriffskontrollen wurden Berechtigungen zentral verwaltet. Bei diesen zentralisierten Modellen existiert ein übergeordneter Mechanismus, der für alle Ressourcen prüft, welche Benutzer Zugriff haben und welche Operationen sie ausführen dürfen.

Der Vorteil liegt hier in der Übersichtlichkeit und der Möglichkeit, Zugriffsrechte an einem Ort konsistent zu steuern, während der Nachteil die potenzielle Komplexität bei wachsenden Benutzer- und Ressourcenzahlen ist.

Die am weitesten verbreiteten zentralisierten Ansätze sind Role-Based Access Control (RBAC) und Attribute-Based Access Control (ABAC).

2.1 Role-based Access-Control (RBAC)

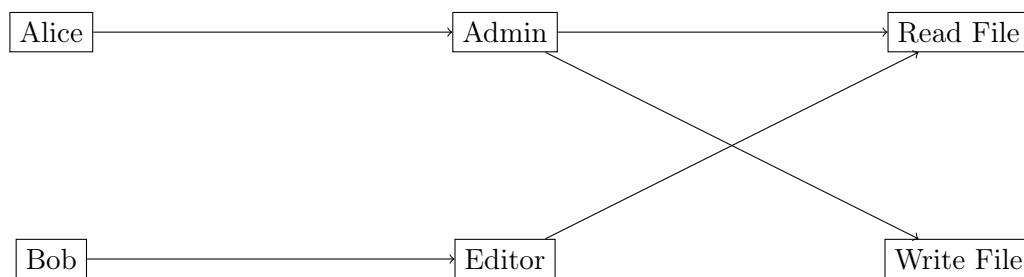


Abbildung 3: Role-Based Access-Control

Bei der rollenbasierten Zugriffskontrolle werden Zugriffsrechte nicht direkt einzelnen Nutzern zugewiesen, sondern Rollen zugeordnet. Nutzer oder Prozesse erhalten anschließend eine oder mehrere Rollen, über die sie die entsprechenden Berechtigungen erhalten [FCK⁺95].

RBAC wird oftmals in Organisationen eingesetzt, in denen Nutzer auf Ressourcen zugreifen müssen, die ihnen nicht selbst gehören. Die Zugriffsberechtigungen ergeben sich aus der Rolle, die ein Nutzer innerhalb der Organisation einnimmt. Somit sind die Nutzer auf die Berechtigungen beschränkt, die ihrer Rolle zugewiesen wurden [FCK⁺95].

Ein typisches Beispiel findet sich in Universitätsinformationssystemen. Dort können Rollen beispielsweise Studierende, Dozierende oder Verwaltungsmitarbeiter sein.

- Studierende können sich für Kurse anmelden, ihre Prüfungsnoten einsehen und auf Bibliotheksressourcen zugreifen.
- Dozenten erstellen Kurse, können Teilnehmerlisten einsehen und Prüfungsleistungen bewerten.

- Verwaltungsmitarbeiter verwalten Gebührenübersichten, Immatrikulationen und Studierendendaten

Ein Vorteil von Rollen-basierter Zugriffskontrolle ist, dass die Zuweisung von Privilegien verständlicher und simpler ist. Man kann etwa die Berechtigungen einer Rolle anpassen, ohne die Berechtigungen aller verknüpften Nutzer verändern zu müssen. Außerdem können Rollen eng der organisatorischen Struktur des jeweiligen Unternehmens folgen [FCK⁺95].

Ein verwandtes Konzept stellt die Access-Control-List (ACL) dar [Bar97]. Bei ACL-basierten Systemen wird für jedes Objekt eine Liste von Nutzern oder Gruppen gespeichert, denen bestimmte Zugriffsrechte gewährt werden. Diese Informationen sind häufig direkt mit dem jeweiligen Objekt, beispielsweise einer Datei, verknüpft. Bei Anfragen überprüft das Betriebssystem anhand dieser Liste, ob der Zugriff erlaubt ist.

2.2 Attribute-based Access-Control (ABAC))

Bei der attributbasierten Zugriffskontrolle werden Zugriffsentscheidungen auf Basis von „Attributen“ getroffen. Zunächst werden Attribute sowie Regeln definiert, die festlegen, unter welchen Bedingungen ein Zugriff erlaubt ist. Diese Regeln werden als „Policies“ bezeichnet. Attribute lassen sich typischerweise in drei Kategorien einteilen [YT05]:

- Subjektattribute beschreiben Eigenschaften von Entitäten wie Nutzer, Anwendungen oder Prozesse, beispielsweise Identität, Rolle oder Organisation.
- Ressourcenattribute beschreiben Eigenschaften von Ressourcen wie Dateien, Datenstrukturen oder Diensten. Beispiele sind Titel, Erstellungsdatum oder Autor eines Dokuments.
- Umgebungsattribute beschreiben den aktuellen Systemkontext, etwa Uhrzeit, Standort oder Netzwerkzustand.

Die definierten Policies werten diese Attribute aus, um zu entscheiden, ob ein Zugriff erlaubt ist. Formal lässt sich eine Policy als eine Funktion beschreiben, die Attribute von Subjekt, Ressource und Umgebung auf eine Zugriffsentscheidung abbildet [YT05].

Ein mögliches Beispiel wäre eine Regel der Form: Ein Nutzer mit der Identität „Julian“ darf auf eine Ressource mit dem Namen „Masterarbeit.tex“ zugreifen, sofern die aktuelle Uhrzeit zwischen „21:00“ und „03:00“ liegt.

Im Gegensatz zur Rollen-basierten Zugriffskontrolle müssen bei ABAC keine festen Rollen definiert und Nutzern zugewiesen werden. Entscheidungen können aufgrund vorhandener Attribute getroffen werden. So können flexible, feingranulare Zugriffsrichtlinien formuliert werden. Insbesondere bei komplexen Systemen mit vielen unterschiedlichen Kontextbedingungen kann ABAC daher eine bessere Skalierbarkeit bieten [YT05].

3 Capability-basierte Zugriffskontrolle

Die zuvor dargestellten Prinzipien wie das Least-Privilege-Prinzip und die strikte Trennung von Zugriffsrechten liefern den theoretischen Rahmen für die Gestaltung sicherer Systeme. Aufbauend auf diesen Grundlagen müssen praktische Mechanismen implementiert werden, die diesen Prinzipien in realen Betriebssystemen und verteilten Umgebungen gerecht werden.

Die Capability-basierte Zugriffskontrolle stellt einen solchen Mechanismus dar, der insbesondere in modernen, verteilten Systemen wie D3OS relevant ist. Sie verfolgt einen dezentralisierten Ansatz, bei dem Berechtigungen nicht über zentrale Listen oder Rollen, sondern als konkrete, nicht manipulierbare Objekte direkt an Prozesse vergeben werden. Diese Objekte, die sogenannten Capabilities, definieren präzise, auf welche Ressourcen und in welcher Weise ein Prozess zugreifen darf.

Durch dieses Modell lassen sich typische Probleme zentraler Zugriffskontrollen vermeiden, wie etwa die versehentliche Eskalation von Rechten durch doppelte Autorität, wie sie beim Confused-Deputy-Problem auftritt. Capabilities ermöglichen eine feingranulare Steuerung von Rechten, unterstützen eine sichere Weitergabe und selektive Entziehung von Berechtigungen und bilden so eine Grundlage für flexible und skalierbare Zugriffskontrollmechanismen in komplexen Systemlandschaften.

Im Folgenden wird die Funktionsweise von Capabilities, deren historischen Ursprünge sowie typische Anwendungsfälle und Limitationen detailliert dargestellt, um die Bedeutung dieses Ansatzes im Vergleich zu traditionellen Methoden zu verdeutlichen.

3.1 Was ist eine Capability-based Access-Control (CapBAC)?

Die Capability-basierte Zugriffskontrolle beruht auf dem Konzept sogenannter Capability-Objekte. Man kann sich diese als Pointer mit besonderen Zugriffsrechten vorstellen [seL25]. Dabei handelt es sich um eindeutig identifizierbare und nicht manipulierbare Schlüssel, die einem Prozess bestimmte Zugriffsrechte auf Ressourcen innerhalb des Betriebssystems gewähren [Han73].

Zur Laufzeit bestimmt die Menge dieser Capability-Objekte, über welche Berechtigungen ein Prozess verfügt, also auf welche Ressourcen er in welcher Weise zugreifen darf. Jede Capability spezifiziert dabei sowohl die betroffene Ressource als auch die erlaubten Zugriffsarten, beispielsweise Lese- oder Schreibzugriffe. Ressourcen können etwa Datenstrukturen, Speichersegmente, IPC-Kanäle oder andere Systemobjekte sein [HRJMS13].

Der Zugriff auf ein durch eine Capability geschütztes Objekt erfolgt ausschließlich unter Vorlage der zugehörigen Capability. Capabilities können dabei weitergegeben („vererbt“) oder entzogen werden und werden in sogenannten Capability-Arrays gespeichert. Der Bereich genauer gesagt die Objekte, die über Capabilities erreicht werden können, wird als Capability-Space bezeichnet [seL25].

Eines der wichtigsten Prinzipien der Capability-Zugriffskontrolle ist das „Principle of least Privilege,, (PoLP). Dieses besagt, dass ein Nutzer maximal eben so viele Rechte erhält, wie für die Ausführung seiner Aufgaben mindestens notwendig ist [BLW⁺24]. Nach diesem Prinzip ist es in der Theorie nicht erforderlich, einmal erteilte Rechte später wieder zu entziehen, da ein Nutzer ohnehin alle Berechtigungen benötigt, die ihm zugewiesen wurden.

Dabei ist es wichtig zwischen einer Speicherverwaltung und einem Zugriffsmanagement zu unterscheiden. Eine Zugriffskontrolle verwaltet Berechtigungen auf Ressourcen, wie etwa Dateien, Speichermedien, Dienste oder Websockets. Sie dient in erster Linie nicht dazu, Speicherzugriffe zu kontrollieren, da sie eine „Ebene“ über einer Speicherverwaltung agiert[WWC⁺14].

3.1.1 Revocation

Auch wenn nach dem Least-Privilege-Prinzip theoretisch kein Entzug notwendig wäre, treten in realen Systemen Situationen auf, in denen Berechtigungen selektiv zurückgenommen werden müssen, auch „Revocation“ genannt. Im Folgenden werden die dafür relevanten Eigenschaften und Mechanismen dargestellt[Gli79]:

1. Selektive: Jeder berechtigte Nutzer/Prozess darf von jeder beliebigen Teilmenge berechtigter Prozesse die Rechte entziehen, denen er zuvor die Rechte zugeteilt hat.
2. Partial: Das Entziehen von Rechten erlaubt es auch, Teile an Berechtigungen zu entziehen. Zum Beispiel nur das Schreiben, nicht das Lesen.

Des Weiteren kann es nützlich sein, ebenfalls die folgenden Eigenschaften zu implementieren [Gli79]:

1. Independent: Wurden zwei Kopien einer Capability aus zwei verschiedenen Quellen erhalten, so können diese Rechte unabhängig voneinander entzogen werden. Hat A etwa von B und C Capabilities geteilt bekommen, kann B nur die B-Capabilities beeinflussen und C lediglich die C-Capabilities.
2. Transitive: Ein Entzug von Rechten propagiert entlang der Kette der geteilten Capabilities. Angenommen A hat Rechte an B geteilt und B an C. Entzieht A dann B die Rechte, so sollten C diese ebenfalls genommen werden. Wäre dies nicht der Fall, könnte C wieder an B teilen, wodurch B diesem Entzug entgehen könnte.
3. Immediate: Nachdem das System den Erhalt des Befehls zum Entziehen einer Berechtigung *registriert* hat, kann der betroffene Prozess nicht mehr auf die Ressourcen zugreifen. Laufende Zugriffe werden hierbei im Laufe der Bearbeitung des Befehls unterbrochen. Dies kann jedoch zu einem inkonsistenten Zustand von Ressourcen führen.

4. Temporal: Es sind auch vorübergehende Revocations möglich. Ein solcher Entzug von Berechtigungen revidiert sich nach einer bestimmten Zeit von selbst. Der Nutzen hierfür ist jedoch gering, da das Teilen, also Wiedererlangen von Rechten simpel ist. Anstelle eines temporalen Entzugs von Berechtigungen könnten diese dauerhaft entzogen und nach der gegebenen Zeit einfach wieder geteilt werden.

3.2 Historie und Entwicklung des Modells

Die Idee der Capabilities hat sich im Laufe der Jahre als Ergänzung zur herkömmlichen Access-Control-List-Methode (kurz: ACL) entwickelt. Obwohl diese neue, innovative Methode zur Zugriffskontrolle vorgestellt wurde, zeigten sich zunächst keine klaren Vorteile gegenüber der ACL. Daher blieben viele weiterhin bei der altbekannten ACL [Har88].

Erst mit dem Auftreten des sogenannten „Confused Deputy“ Problems, wurde ein großer Vorteil der Capabilities über die bewährten Methoden erkannt.

3.2.1 Das Confused-Deputy Problem

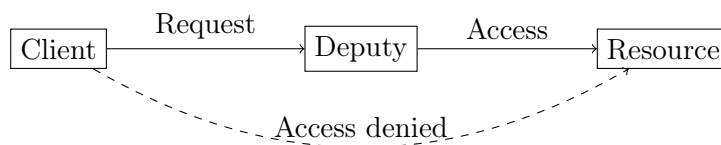


Abbildung 4: Confused Deputy Problem

In der ersten Version dieses Problems ([Wik25c], [Har88]) existiert ein Compiler im Verzeichnis SYSX, dem Homeverzeichnis eines UNIX-ähnlichen Betriebssystems. Ein Nutzer kann diesen Compiler aufrufen und ihm den Namen einer Datei übergeben, in die zusätzlicher Debugging-Output geschrieben wird.

Damit der Compiler zusätzliche rein statistische Informationen in einer Datei in dem Home-Verzeichnis des Betriebssystems ablegen kann, wird ihm eine sogenannte „home files licence“ zugewiesen, sodass er diese Zugriffe durchführen kann. Im selben Verzeichnis befindet sich zudem eine Datei mit wichtigen Abrechnungsinformationen, auf die nur mit der entsprechenden Lizenz zugegriffen werden darf [Har88].

Das Problem besteht nun darin, dass ein Nutzer, der zwar keinen direkten Zugriff auf das Home-Verzeichnis besitzt, jedoch den Compiler aufrufen kann, diesem die Abrechnungsdatei als Ziel für den Debugging-Output angeben kann. Der Compiler wendet sich dann an das Betriebssystem mit der Anfrage, seinen Output in die Abrechnungsdatei zu schreiben. Dies wird ihm gewährt, da ihm die erforderliche „home file licence“ zugewiesen wurde [Har88].

Also löscht der Compiler auf Bitte eines dazu unberechtigten Nutzers hin die Abrechnungs-

informationen vollständig, da der Compiler die für diesen Zugriff notwendige Lizenz besitzt [Har88].

3.2.2 Limitierungen unter ACL

Unter der zuvor verwendeten Access-Control-List ist es nicht möglich, dieses Problem elegant zu lösen [Har88].

Es ist nicht notwendig für den Compiler zu überprüfen, ob die Output Datei in einem anderen Verzeichnis liegt, da dies ein völlig normaler use-case zur Zeit seiner Entwicklung ist. Ebenfalls ist es nicht notwendig zu prüfen, ob die Output-Datei im Homeverzeichnis landet, da ein Nutzer mit vorhandenen Berechtigungen eventuell ein legitimes Interesse daran hat, die Datei dort abzulegen. Auch auf einen bestimmten Dateinamen (hier den der Abrechnungsdatei) zu prüfen ist nicht sinnvoll, da in dem Verzeichnis ebenfalls noch andere wichtige Dateien liegen. Man müsste anderenfalls bei jeder neu hinzugefügten Datei den Compiler abändern [Har88].

Der Compiler funktioniert seit seiner Entwicklung fehlerfrei. Erst durch die zugewiesene Berechtigung wurde sein Verhalten fehlerhaft [Har88].

Das Problem ist also nicht der Compiler selbst, sondern entsteht durch die „doppelte Autorität“. Der Nutzer tritt dem Compiler seine Autorität ab, indem er ihn aufruft und ihn den Debugging Output ausgeben lässt. Der Compiler besitzt zusätzlich die Autorität auf das Homeverzeichnis zuzugreifen, da er den Statistik-Output schreiben soll [Har88].

Er selbst kann hier nicht zwischen den beiden Berechtigungen unterscheiden, da er keine Möglichkeit besitzt, der Zugriffskontrolle auszudrücken, welchen Zugriff er mit welcher erteilten Berechtigung durchführen möchte [Har88].

Eine Lösung wäre ein System, unter welchem der Compiler zwischen seinen beiden Autoritäten hin und her wechseln könnte. Dies erhöht jedoch die Komplexität sowohl des Compilers als auch der Zugriffskontrolle. Es könnte für andere Programme unter einem solchen System auch notwendig sein, zwischen mehr als nur zwei Autoritäten hin und her zu wechseln. Darüber hinaus existieren mehr Zugriffskontrollmechanismen als nur die im Bezug auf Dateien.

Ein genereller Work-Around ist also mit der ACL nicht möglich [Har88].

3.2.3 Die Lösung

Wird eine Capability-basierte Zugriffskontrolle eingesetzt, erhält der Compiler weiterhin Zugriff auf die Statistikdatei, jedoch nicht mehr in Form einer globalen Berechtigung, sondern als spezifische Capability. Zugriffe erfolgen dabei nicht mehr über Dateinamen oder globale Namensräume, sondern ausschließlich über die jeweils verwendete Capability.

Für Schreibzugriffe auf die Statistikdatei nutzt der Compiler somit ausschließlich seine fest zugewiesene Statistik-Capability.

Für die Ausgabe des Debug-Outputs erhält der Compiler beim Aufruf durch den Nutzer eine separate Capability, die genau die Zielfile repräsentiert. Diese wird vom Nutzer bereitgestellt und entspricht genau den Rechten, über die der Nutzer selbst verfügt.

Dadurch ergibt sich eine klare Trennung der Autoritäten: Der Compiler kann den Debug-Output ausschließlich über die vom Nutzer übergebene Capability schreiben und hat keine Möglichkeit, stattdessen seine eigene Statistik-Capability zu verwenden. Umgekehrt kann er den Statistik-Output nur in die Statistikdatei schreiben.

Auf diese Weise ist sichergestellt, dass der Debug-Output nur in Dateien geschrieben werden kann, für die der aufrufende Nutzer autorisiert ist, während gleichzeitig verhindert wird, dass privilegierte interne Rechte des Compilers für fremde Zwecke missbraucht werden [\[Har88\]](#).

4 Existierende Systeme

Um eine strukturierte und sichere Implementierung einer Capability-basierten Zugriffskontrolle zu entwickeln, ist zunächst eine Analyse bestehender Systeme sinnvoll. Insbesondere Betriebssysteme und Sicherheitsarchitekturen, die bereits Capability-basierte Modelle einsetzen, liefern wichtige Designentscheidungen und Implementierungsstrategien.

Im Folgenden wird eine Auswahl der für diese Arbeit relevanten Betriebssysteme und APIs vorgestellt.

4.1 L4

L4, genauer der L4/x86 Mikrokernel [Lie96], wurde von Jochen Liedtke als Nachfolger des L3-Kernels entwickelt. Die von L4 definierte ABI bildete später die Grundlage für eine Reihe von Mikrokernel- und Hypervisor-Systemen, darunter seL4 und NOVA.

Der ursprüngliche L4-Kernel selbst implementierte noch kein Capability-basiertes Zugriffskontrollmodell. Eine entsprechende Erweiterung wurde erst mit OKL4 im Jahr 2007 innerhalb der L4-Familie eingeführt [HE16].

4.2 seL4

Der seL4-Kernel [Hei20] („secure embedded L4“ [Wik25a]) ist ein Mikrokernel und Hypervisor, der aus der L4-Familie hervorgegangen ist. Eine Besonderheit von seL4 ist, dass sowohl die funktionale Korrektheit der Implementierung als auch die zentralen Sicherheitseigenschaften formal mathematisch bewiesen und maschinell verifiziert wurden.

Für diese Arbeit ist besonders relevant, dass seL4 vollständig auf Capability-basierter Zugriffskontrolle basiert [Hei20]. Zugriffe auf Kernelobjekte erfolgen ausschließlich über Capabilities.

In seL4 werden Capabilities in sogenannten Capability Spaces (CSpaces) gespeichert. Jeder Prozess besitzt einen eigenen Capability-Space, der als strukturierte Sammlung von Capability-Slots organisiert ist. Beim Zugriff auf ein Kernelobjekt wird die entsprechende Capability über ihre Position im Capability-Space referenziert.

4.3 NOVA

NOVA [SK10] ist ein Mikrokernel-Hypervisor. Die Architektur verfolgt ähnliche Sicherheitsziele wie seL4, insbesondere eine starke Isolation zwischen den Komponenten.

Auch NOVA verwendet ein Capability-basiertes Zugriffskontrollmodell. Die grundlegende

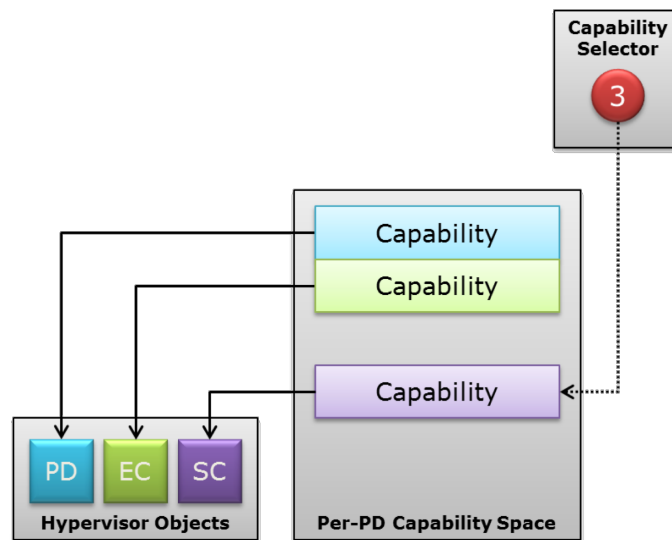


Abbildung 5: Capability Spaces in Nova [Ste13]

Idee ähnelt dabei dem Ansatz von seL4, bei dem Zugriffe auf Kernelobjekte ausschließlich über Capabilities erfolgen.

4.4 Mungi

Das Betriebssystem MungiOS [HEV⁺98] verfolgt ein Single-Address-Space-Modell. Im Gegensatz zu klassischen Betriebssystemen besitzen dabei alle Anwendungen einen gemeinsamen virtuellen Adressraum.

In traditionellen Betriebssystemen können Speichergrenzen zwischen Prozessen nur durch Kernelmechanismen überwunden werden. In Mungi ist der Zugriff auf Speicherbereiche hingegen über Capabilities geregelt.

Capabilities werden hierbei als geschützte Nutzerobjekte implementiert. Ihre Gültigkeit wird bei Zugriffen anhand einer zentralen Tabelle überprüft, welche die Capability-Identifikatoren mit zugehörigen Passwörtern verknüpft.

4.5 J-Kernel

Das Betriebssystem J-Kernel [ECC⁺99] implementiert einen Capability-basierten Zugriffskontrollansatz auf Basis der Java Virtual Machine. Hierbei werden Java-Objektreferenzen genutzt, um eine Capability-Semantik abzubilden.

Hierzu werden sogenannte „Tasks“, wie Prozesse in sogenannten „protection domains“ voneinander abgeschottet. Zwischen diesen Domains können ausschließlich kontrollierte

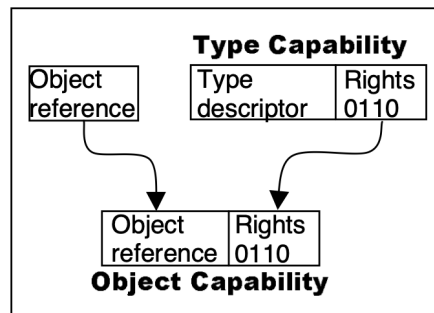


Abbildung 6: Java-Objektreferenzen mit Zugriffsrechten kombinieren[HCC⁺97]

Referenzen auf Capability-Objekte geteilt werden, um so das Entziehen von Berechtigungen ohne zusätzlichen Overhead für andere Objektzugriffe zu ermöglichen.

J-Kernel ist für diese Arbeit insbesondere deshalb relevant, weil ein Teil der Sicherheitsgarantien durch Eigenschaften der Programmiersprache selbst erreicht wird. Die Typsicherheit von Java stellt sicher, dass Referenzen nicht manipuliert werden können. Ein ähnlicher Gedanke liegt auch der Wahl von Rust als Implementierungssprache für D3OS zugrunde, da Rust durch sein Typsystem und das Borrow-Checking zusätzliche Sicherheitsgarantien bietet.

4.6 Capsicum

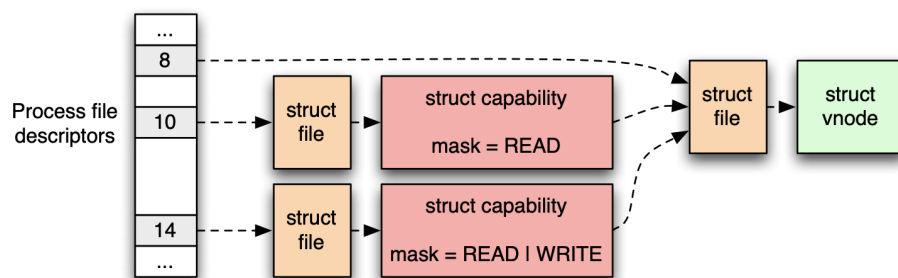


Abbildung 7: Capability-Modus zu Unix-Dateideskriptoren hinzufügen[WALK10]

Capsicum [WALK10] ist eine Capability-basierte Sicherheits-API für UNIX-Systeme. Ziel ist es, Anwendungen innerhalb bestehender UNIX-Systeme stärker zu isolieren und das Prinzip der geringsten Privilegien umzusetzen.

Hierzu erweitert Capsicum bestehende UNIX-Mechanismen, insbesondere Filedeskriptoren, um einen sogenannten Capability Mode. In diesem Modus können Prozesse nur noch auf

Ressourcen zugreifen, für die sie bereits entsprechende Deskriptoren besitzen.

Dadurch lassen sich Anwendungen in isolierte Komponenten aufteilen, die jeweils nur über die minimal erforderlichen Berechtigungen verfügen.

Für diese Arbeit ist Capsicum insbesondere deshalb relevant, weil es zeigt, wie Capability-basierte Konzepte in bestehende Systemarchitekturen integriert werden können. Capsicum-Capabilities entsprechen dabei nicht vollständig klassischen Capability-Systemen, sondern stellen eher eine Einschränkung bestehender Zugriffsmöglichkeiten dar.

4.7 Google Fuchsia

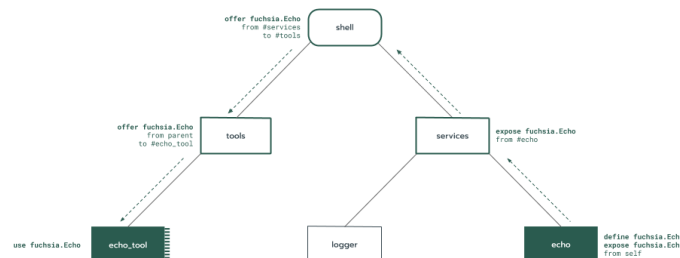


Abbildung 8: Vererbungshierarchie[Goo25]

Das Betriebssystem Fuchsia [Goo25] basiert auf dem Zircon-Kernel, der ebenfalls ein Capability-basiertes Zugriffskontrollmodell implementiert.

Zentrale Kommunikationsmechanismen zwischen Systemkomponenten erfolgen über sogenannte „Channels“. Der Zugriff auf diese Kommunikationsobjekte wird über Capabilities geregelt.

Der Kernel stellt sicher, dass Kommunikationsbeziehungen kontrolliert aufgebaut werden und keine unerwünschten Abhängigkeitsstrukturen entstehen.

4.8 EROS

EROS [SSF99] ist ein Mikrokern-Betriebssystem, das vollständig auf Capability-basierter Zugriffskontrolle entwickelt wurde.

Das System besteht aus voneinander isolierten Komponenten, die jeweils in eigenen Protection Domains ausgeführt werden. Eine Protection Domain wird durch die Menge der Capabilities definiert, über die eine Komponente verfügt.

EROS integriert außerdem persistente Speichermechanismen direkt in das Capability-Modell. Ressourcen, die in klassischen UNIX-Systemen als Dateien dargestellt werden, werden in EROS ebenfalls über Capabilities referenziert.

4.9 Relevanz der betrachteten Systeme für D3OS

Die Analyse bestehender Capability-basierter Systeme zeigt unterschiedliche Ansätze zur Umsetzung von Zugriffskontrolle. Für die Entwicklung in D3OS sind insbesondere einige Konzepte aus seL4, NOVA, J-Kernel, Fuchsia und EROS relevant.

Die Architektur von seL4 und NOVA dient als zentrale Orientierung für das grundlegende Capability-Modell. Beide Systeme organisieren Capabilities in prozessspezifischen Capability-Spaces und erzwingen, dass Zugriffe auf Kernelobjekte ausschließlich über Capabilities erfolgen. Dieses Prinzip wird auch für D3OS übernommen. Kernelressourcen können nur über entsprechende Capabilities adressiert werden. Systemaufrufe werden daher so gestaltet, dass sie ausschließlich Capability-Referenzen als Zugriffsmittel verwenden. Die eigentlichen Capabilities verbleiben dabei im Kernel, während Nutzerprogramme nur indirekt über Referenzen auf diese zugreifen.

Ein weiterer relevanter Aspekt ist die Nutzung der Typsicherheit der Programmiersprache zur Unterstützung der Zugriffssicherheit. Das J-Kernel-System zeigt, dass Sprachmechanismen wie typsichere Objektreferenzen zur Absicherung von Capabilities beitragen können. In D3OS kann ein ähnlicher Effekt durch die Eigenschaften der Programmiersprache Rust erreicht werden. Insbesondere das Typsystem, sowie Ownership-Regeln als auch Lifetimes können genutzt werden, um Fehlverwendungen von Capability-Referenzen zu verhindern.

Das Betriebssystem Fuchsia liefert darüber hinaus ein interessantes Modell für die Weitergabe von Capabilities. Dort entsteht durch die Weitergabe von Ressourcen eine baumartige Struktur von Abhängigkeiten zwischen Komponenten. Dieser Aufbau verhindert zyklische Interdependenz zwischen Ressourcen und sorgt für eine klare Parent-Child-Beziehung zwischen delegierten Rechten. Ein ähnliches Prinzip kann auch für D3OS genutzt werden, um die Delegation von Capabilities strukturiert zu kontrollieren.

Das Capability-Modell von EROS zeigt zudem, wie persistente Ressourcen wie Dateien vollständig in ein Capability-basiertes Zugriffssystem integriert werden können. In EROS werden Dateioperationen ausschließlich über Capabilities ausgeführt. Dieses Prinzip ergänzt den Ansatz aus seL4, da es verdeutlicht, wie auch klassische Betriebssystemressourcen in ein einheitliches Capability-Modell integriert werden können.

Andere betrachtete Systeme sind für die Architektur von D3OS hingegen weniger relevant. Mungi basiert auf einem Single-Address-Space-Modell und verwendet Capabilities teilweise als Nutzerobjekte im Userspace. Da D3OS kein Single-Address-Space-Betriebssystem ist und Capabilities ausschließlich im Kernel verwaltet werden sollen, sind die dort beschriebenen Mechanismen für den Schutz von Userspace-Capabilities für diese Arbeit nicht relevant.

Capsicum verfolgt einen anderen Ansatz, bei dem bestehende UNIX-Zugriffsmechanismen um capability-ähnliche Einschränkungen erweitert werden. Ziel dieser Arbeit ist jedoch die Implementierung eines vollständig Capability-basierten Zugriffssystems und nicht lediglich die Erweiterung eines bestehenden Zugriffskontrollmodells. Die in Capsicum beschriebenen

Mechanismen sind daher nur begrenzt auf D3OS übertragbar.

Zusammenfassend orientiert sich die in dieser Arbeit entwickelte Zugriffskontrolle primär an den Capability-Konzepten von seL4 und NOVA, ergänzt durch strukturelle Ansätze aus Fuchsia sowie Integrationskonzepte aus EROS. Sprachbasierte Sicherheitsmechanismen, wie sie im J-Kernel diskutiert werden, können zusätzlich durch die Eigenschaften der Programmiersprache Rust unterstützt werden.

5 Implementierung

Dieses Kapitel beschreibt die Implementierung der Capability-basierten Zugriffskontrolle in D3OS. Ziel ist es, den Zugriff auf Systemressourcen strikt über explizit vergebene Berechtigungen zu steuern, sodass Prozesse ausschließlich auf Ressourcen zugreifen können, für die sie autorisiert sind.

Zunächst wird das grundlegende Konzept der Capabilities eingeführt. Darauf aufbauend wird der Capability-Space (CSpace) als zentrale Verwaltungsstruktur aller Capabilities eines Prozesses erläutert. Anschließend wird die Integration in Kernel und User-Library beschrieben, einschließlich der Mechanismen für Threads, Systemaufrufe und das Naming-System. Abschließend werden Prozesse zur Weitergabe (Share) und zum Entzug (Revocation) von Capabilities betrachtet.

Die Darstellung folgt dabei einem strukturierten Aufbau von der internen Datenstruktur über Prozess- und Thread-Management bis hin zur systemweiten Zugriffskontrolle.

5.1 Capability

Quellcode 1: Capability Struct (capability.rs)

```

1: pub struct Capability<T> {
2:     obj: Option<Arc<Mutex<T>>>,
3:     flags: CapabilityFlags,
4:     shared_to: Mutex<Vec<Weak<Capability<T>>>>,
5:     original: bool,
6: }
```

Die Capabilities wurden als generisch typisierte Container implementiert, die sowohl das geschützte Objekt als auch die zugehörigen Zugriffsrechte kapseln. Siehe Quellcode 1.

Zum einen speichert das Feld `flags` die mit der Capability verbundenen Berechtigungen. Diese werden durch `CapabilityFlags` repräsentiert und definieren, ob ein Objekt gelesen (READ) geschrieben (WRITE) oder die Capability selbst weitergegeben werden darf (SHARE).

Das eigentliche Objekt wird im Feld `obj` gespeichert. Dieses Objekt wird in mehreren Datenstrukturen gekapselt, um sichere und parallele Zugriffe zu ermöglichen. Es wird zunächst in einem `Option`-Container gespeichert. Dadurch kann eine Capability durch einen Revocation-Vorgang ungültig gemacht werden, indem das enthaltene Objekt durch `None` ersetzt wird. Ein Zugriff auf die Capability führt in diesem Fall zu einem Fehler. Zusätzlich wird das Objekt in `Arc<Mutex<T>` gekapselt. Der `Arc` ermöglicht dabei die gemeinsame Nutzung desselben Objekts zwischen mehreren Threads, während der `Mutex` eine synchrone, threadsichere Modifikation gewährleistet.

Die Capability speichert außerdem Informationen über Delegationen. Über die Struktur `shared_to` werden schwache Referenzen auf Capabilities gesichert, an die diese Capability weitergegeben wurde. Dadurch kann bei einem Revocation-Vorgang nachvollzogen werden, welche weiteren Capabilities ebenfalls betroffen sind.

5.2 Capability-Space

Quellcode 2: CSpace Struct (cspace.rs)

```

1: pub struct CSpace{
2:     syscall_capabilities: Vec<Capability<Syscall>>,
3:     naming_capabilities: Vec<Capability<NamingObject>>,
4:     open_naming_capabilities: Vec<Capability<NamingObject>>,
5:     //... other capability types
6: }
```

Die zuvor beschriebenen Capability Objekte werden in sogenannten Capability-Spaces (kurz: CSpace) organisiert. Dieses Konzept orientiert sich am Design des seL4-Kernels [seL25], in dem CSspaces ebenfalls als eigene Kernel-Objekte über Capabilities referenziert werden.

Ein CSpace stellt dabei eine Sammlung aller Capabilities dar, die einem Prozess zur Verfügung stehen. In der aktuellen Implementierung wird ein CSpace als Datenstruktur implementiert, welche mehrere Listen von Capabilities enthält. Diese Listen sind nach Capability-Typen getrennt, beispielsweise für Systemaufrufe oder Naming-Objekte. Die Trennung erzwingt durch Rusts Typsystem eine typsichere und klar strukturierte Verwaltung der Capabilities. Der Zugriff auf eine Capability erfolgt über ihre Position innerhalb der jeweiligen Liste.

Grundsätzlich existieren zwei Ansätze zur Speicherung von Capabilities.

Die erste Möglichkeit besteht darin, Capabilities im Userspace zu halten. Dieser Ansatz birgt jedoch das Risiko, dass ein Nutzer Capabilities manipuliert, um unautorisiert auf geschützte Ressourcen zuzugreifen. Um dies zu verhindern, müssten Capabilities kryptographisch abgesichert werden, etwa durch digitale Signaturen. Der Kernel wäre in diesem Fall gezwungen, bei jedem Zugriff die Integrität der Capability zu überprüfen. Auch bei der Weitergabe von Capabilities müssten neue Signaturen erzeugt werden, was zusätzlichen Aufwand verursacht.

Ein Beispiel für diesen Ansatz ist das Betriebssystem Mungi. Dort werden Capabilities als Userspace-Objekte gespeichert, deren Integrität durch zentrale Tabellen abgesichert wird, die Capability-Identifizierer mit geheimen Passwörtern verknüpfen. Bei jedem Zugriff wird die übergebene Capability anhand dieser Tabelle validiert [HEV⁺98].

Die zweite Möglichkeit basiert auf einer Indirektion, bei der Capabilities vollständig im Kernel verwaltet werden.

Dieser Ansatz stellt sicher, dass Anwendungen ihre eigenen Berechtigungen nicht unautorisiert verändern können, da sämtliche Zugriffe kontrolliert über den Kernel erfolgen. Die Anwendungen halten lediglich Referenzen auf die Positionen der im Kernel gespeicherten Capabilities.

Bei einem Zugriff übergibt die Anwendung dem Kernel eine Referenz auf die gewünschte Capability. Der Kernel löst diese auf und führt die entsprechende Operation aus.

Die Weitergabe von Rechten erfolgt analog. Der Kernel erhält die Information, welche Capability übertragen werden soll, und trägt diese direkt in den CSpace des Zielprozesses ein [seL25].

In dieser Arbeit wird der In-Kernel-Ansatz verwendet. Da dies dem Open-Design Kriterium entspricht und so analog zu dem verifizierten Mikrokernel seL4 entwickelt wird.

5.2.1 Kernel-Integration

Quellcode 3: Prozess (process.rs)

```

1: pub struct Process {
2:     pub id: usize,
3:     pub virtual_address_space: VirtualAddressSpace,
4:     pub cspace: Capability<CSpace>,
5: }

```

Ein CSpace wird automatisch bei der Erstellung eines Prozesses erzeugt. Der Prozess erhält hierbei analog zu seL4 [seL25] eine Capability, die auf den CSpace verweist, siehe Quellcode 3.

Die Capability wird verwendet, da Threads 4 eines Prozesses nicht etwa einen eigenen CSpace erhalten, sondern im Regelfall den des Prozesses übernehmen, siehe Abbildung 9. Dadurch besitzen alle Threads eines Prozesses dieselben Berechtigungen und können gemeinsam auf Ressourcen zugreifen.

Quellcode 4: Thread (thread.rs)

```

1: pub struct Thread {
2:     id: usize,
3:     stacks: Mutex<Stacks>,
4:     process: Arc<Process>, // reference to my process
5:     pub(crate) cspace: Capability<CSpace>, // capability to CSpace
6:     /// for user threads: the address to jump to
7:     user_kickoff: VirtAddr,
8:     /// the actual entry point
9:     entry: extern "sysv64" fn(),
10: }

```

Dieses Verhalten erleichtert die Implementierung von Multithreading innerhalb einer Anwendung. Wird beispielsweise eine neue Capability erzeugt oder empfangen, können alle Threads des Prozesses unmittelbar darauf zugreifen.

Es ist prinzipiell möglich, einen Thread mit einem eigenen, leeren CSpace zu erzeugen. Daher werden Operationen wie Share und Revoke auch auf Thread- und nicht auf Prozessebene durchgeführt. Jedoch ist dies selten notwendig, da hierbei keine wahre Isolation gewährleistet wird, die die Verwendung eines neuen Prozesses an dieser Stelle bieten würde, da Threads weiterhin denselben Adressraum und Kernel-Kontext teilen und somit keine vollständige Isolation gewährleisten. Prozesse sind eben dafür gedacht, den Ressourcenzugriff zu gruppieren und stellen zusammenhängende „Protection-Domains“ dar [TB15].

Windows und Unix verwenden einen solchen Mechanismus jedoch tatsächlich. Hier als „Impersonation“ bezeichnet, werden Threads mit abweichenden Rechten innerhalb eines Prozesses verwendet, um im Namen anderer Nutzer Zugriffe durchzuführen [Ka01].

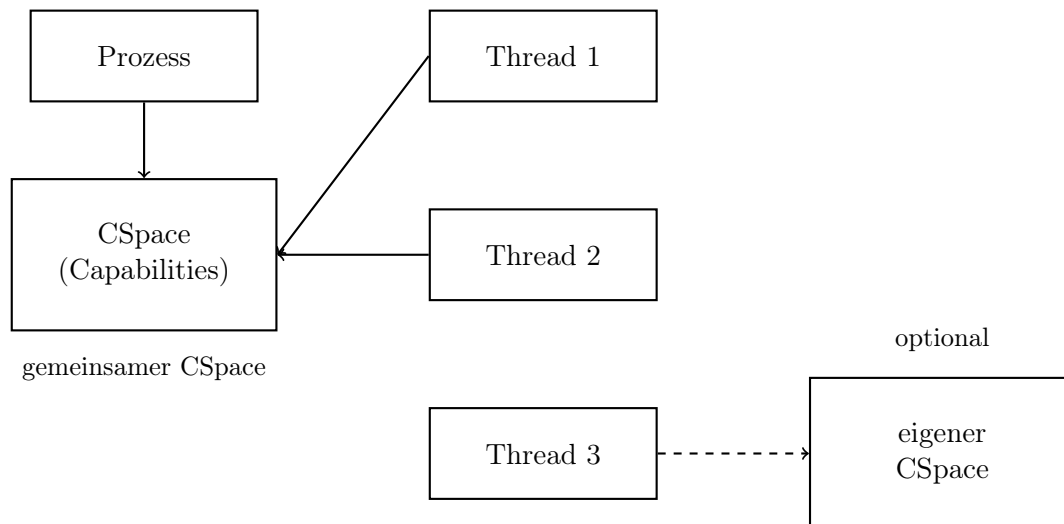


Abbildung 9: CSpace Konfiguration

Während der initialen Erstellung des CSpaces werden Capabilities für alle verfügbaren Systemaufrufe und ausgewählte Ressourcen erzeugt und abgelegt. Siehe Quellcode 5 für den genauen Ablauf.

Für eine feingranulare Rechtevergabe sollte der CSpace initial nur minimal ausgestattet werden. Erweiterte Rechte würden anschließend explizit durch den erzeugenden Prozess übertragen. Um die weitere Entwicklung zu erleichtern, wurde dies für einen späteren Zeitpunkt aufgespart.

Zu den vergebenen Ressourcen zählen insbesondere das Root-Verzeichnis des Dateisystems sowie eine öffentliche Pipe zur Kommunikation zwischen Threads.

Die Capability für das Root-Verzeichnis ist erforderlich, um neue Dateien oder Verzeichnisse

anzulegen. Beim Erzeugen eines Objekts muss stets eine Capability auf das übergeordnete Verzeichnis übergeben werden. Da der Zugriff auf einzelne Dateien wie in EROS [SSF99], weiterhin ausschließlich über deren spezifische Capabilities erfolgt, entsteht dadurch keine zusätzliche Sicherheitslücke. Nur durch Zugriff auf den Ordner, werden eben nicht direkt auch die enthaltenen Dateien offengelegt.

Die initiale Ausstattung eines CSpace kann grundsätzlich pro Prozess durch die erzeugende Anwendung konfiguriert werden. Diese Funktionalität ist in der aktuellen Implementierung noch nicht vorgesehen, kann jedoch zu einem späteren Zeitpunkt ergänzt werden.

Eine öffentliche Pipe wird derzeit verwendet, um Threads über neu erhaltene Capabilities zu informieren. Dieses Verfahren ist jedoch unsicher, da Nachrichten von beliebigen Anwendungen mitgelesen werden können. Perspektivisch sollte diese Kommunikation durch ein sicheres Message-Passing-System ersetzt werden. Ein solches System ist in D3OS aktuell nicht implementiert (siehe Kapitel 7.1).

Quellcode 5: Initialize CSpace (cspace.rs)

```

1: pub fn new() -> Self {
2:     let syscall_fns: [*const (); NUM_SYSCALLS] = [
3:         sys_terminal_read as *const (), //0
4:         sys_terminal_read_nb as *const (),
5:         sys_terminal_write as *const (),
6:         //...
7:         sys_share_naming_cap as *const (), //38
8:     ];
9:
10:    let mut num = 0;
11:    let mut syscall_capabilities: Vec<_> = syscall_fns
12:        .iter()
13:        .map(|&f| {
14:            let cap = Capability::syscall(
15:                Syscall::new(num, f));
16:            num += 1;
17:            cap
18:        })
19:        .collect();
20:
21:    let mut naming_capabilities = Vec::new();
22:    //check if naming is initialized already
23:    if api::ROOT.is_completed() {
24:        if let Some(root) = api::ROOT.get(){
25:            let root_cap = create_naming_capability(
26:                as_named_object(root.root_dir()),
27:                OpenOptions::all(),
28:                None);
29:            let shared_pipe = shared_pipe(&root_cap);
30:            naming_capabilities.push(root_cap);
31:            naming_capabilities.push(shared_pipe);

```

```

32:         }
33:     }
34:
35:     Self {
36:         syscall_capabilities,
37:         naming_capabilities,
38:         //memory_capabilities: Vec::new(),
39:         //driver_capabilities: Vec::new(),
40:         //... initialize other capability types
41:     }
42: }

```

5.2.2 User-API

Die User-Library wurde so angepasst, dass die kernelinterne Capability-basierte Zugriffskontrolle für Anwendungen möglichst einfach nutzbar ist.

Systemaufrufe erfolgen weiterhin über Library-Funktionen. Diese übergeben jedoch nicht mehr direkt eine klassische Syscall-Nummer, sondern adressieren Capabilities im CSpace des Prozesses. Die Library kapselt diesen Mechanismus vollständig ab, sodass sich aus Anwendungssicht das Aufrufschema weitgehend nicht verändert.

Auch das Naming-System wurde grundlegend überarbeitet.

Zuvor erfolgten die Erstellung und der Zugriff auf Dateien über Pfadangaben. Beim Öffnen wurde ein Eintrag in der Open-Objects-Tabelle erzeugt, und der Nutzer erhielt ein Handle auf diesen.

Quellcode 6: User Capability (/library/capabilities/src/capability.rs)

```

1: pub struct Capability{
2:     handle: usize,
3: }

```

Im Capability-basierten System werden Naming-Objekte nicht mehr über Pfade allein adressiert, sondern über ihren Namen in Kombination mit einer Capability auf das übergeordnete Verzeichnis. Diese Capability existiert im User-Mode und enthält lediglich ein Handle auf die entsprechende Kernel-Struktur im CSpace. Siehe Quellcode 6.

Weitere Zugriffe erfolgen ausschließlich über diese Capability. Beispielsweise wird sie benötigt, um innerhalb eines Verzeichnisses neue Objekte zu erzeugen. Beim Öffnen einer Datei wird weiterhin ein Handle zurückgegeben. Dieses referenziert jedoch nun eine speziell erzeugte Open-Capability, über die Lese- und Schreiboperationen durchgeführt werden.

5.3 Syscall-Management

Eine zentrale Ebene der Zugriffskontrolle besteht in der Absicherung von Systemaufrufen („Syscalls“). Hierzu ist für das bestehende System eine Änderung notwendig.

In der ursprünglichen Implementierung wurden Systemaufrufe über eine Syscall-Nummer adressiert. Diese Nummer wurde im Register `rax` übergeben, während weitere Parameter über zusätzliche Register (`rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`) übertragen wurden. Der Kernel überprüfte anschließend die Syscall-Tabelle und führte die entsprechende Funktion aus.

Im Capability-basierten Modell wird statt der Syscall-Nummer ein Index auf eine Capability im CSpace übergeben. Der Kernel sichert weiterhin die Register, lädt jedoch anschließend die Capability aus dem CSpace und führt die darin referenzierte Operation aus.

Ist an der angegebenen Position keine gültige Capability vorhanden, etwa weil sie entzogen wurde oder nie existierte, wird der Aufruf abgebrochen und ein Fehlercode zurückgegeben. Auf diese Weise können Systemaufrufe nur noch durchgeführt werden, wenn der aufrufende Prozess über die entsprechende Capability verfügt.

5.4 Naming-System

Neben den Syscalls wurde auch das Naming-System für den Capability-Support überarbeitet. Dies ist insbesondere notwendig, da es momentan die einzige Möglichkeit für Inter-Prozess-Kommunikation zur Verfügung stellt.

Auch hier waren grundlegende Veränderungen notwendig.

5.4.1 Bisherige Implementierung

Das ursprüngliche Betriebssystem implementierte eine Open-Objects-Tabelle für den Umgang mit Naming-Objekten. Naming-Objekte sind etwa Dateien, Ordner, aber auch Pipes für die Verständigung zwischen Threads.

Beim Erstellen wurde ein Objekt unter einem Pfad im Dateisystem abgelegt. Für den Zugriff musste es geöffnet werden. Hierfür wird in der Open-Objects-Tabelle ein Naming-Objekt abgelegt und dem Nutzer die Position, also ein „Handle“ mitgeteilt. Dieser übersendet für Befehle wie Lesen, Schreiben oder ähnlichen dieses Handle, um den Kernel zu informieren, auf welches Objekt zugegriffen werden soll. Nach den Zugriffen muss das Objekt dann „geschlossen“ werden, also aus der Tabelle entfernt werden.

5.4.2 Capability Support

In der Capability-Implementierung entfällt die Open-Objects-Tabelle. Stattdessen wird beim Erstellen eines Naming-Objekts direkt eine Capability erzeugt und im CSpace des Prozesses abgelegt. Der Nutzer erhält das zugehörige Handle. Alle Zugriffe außer Lesen und Schreiben erfolgen direkt über diese Capability.

Beim Öffnen wird eine zusätzliche Capability erzeugt und im Bereich „open_naming_objects“ des CSpace abgelegt. Diese wird für Lese- und Schreibzugriffe verwendet. Der Grund ist, dass blockierende Operationen auf der ursprünglichen Capability andere Prozesse behindern würden, die dieselbe Capability nutzen. Durch diese Trennung wird die ursprüngliche Capability nicht blockiert.

Da Zugriffe weiter nur über ein Handle erfolgen, bleibt die Handhabung aus Nutzersicht identisch.

Im Falle des Erstellens einer Pipe geschieht dies analog zu Quellcode 7. Hier wird eine Pipe „revoke“ im Root-Ordner erstellt. Dies entspricht dem Pfad „/revoke“ im vorherigen System.

Quellcode 7: Pipe Erstellen durch User

```
1: mkfifo("revoke", OpenOptions::all(), ROOT)
```

Ein wesentlicher Unterschied besteht darin, dass Zugriffe nur noch auf vorhandene Capabilities möglich sind. Dies schränkt Funktionen wie zum Beispiel `ls` ein, da nicht unbedingt alle Dateien in einem Ordner sichtbar sind. Die aktuelle Implementierung dieser Funktionen stellt daher einen Bruch in der Zugriffskontrolle dar und wurde entfernt. Zugriffe können nur noch, auf Objekte erfolgen, die selbst erstellt wurden, oder die von anderen Prozessen mit dem Aufrufer geteilt werden.

Anders als bei den Systemaufrufen ist es im Naming-System nicht notwendig, die Capabilities in einer bestimmten Reihenfolge festzuhalten. Neue Naming-Capabilities werden also simpel in die CSpace-Liste angehängen und bei Entzug der Rechte durch Null-Capabilities ersetzt.

Dies hinterlässt auf Dauer zwar blockierte Plätze im Anwendungs-CSpace, jedoch ist dies zwingend notwendig. Würden die Slots, ohne die Anwendung zu informieren mit neuen Capabilities belegt werden, so würde diese unter Angabe des Platzes in der Rechte-Liste Aufrufe auf die falsche Datei durchführen. Es wäre möglich hier eine Funktion hinzuzufügen, über die eine Anwendung nach einem fehlgeschlagenen Aufruf die betroffene Capability löscht. All dies sollte jedoch nicht zu Problemen führen, da die Liste über neun Trillionen Plätze pro Anwendung unterstützt und nach der Capability-Philosophie Revokes eher selten auftreten.

5.5 Share-Mechanismus

Quellcode 8: Share (capability.rs)

```

1: pub fn share(&self, new_flags: CapabilityFlags)
2:     -> Option<Capability<T>> {
3:     if !self.has_permissions(CapabilityFlags::SHARE) {
4:         return None;
5:     }
6:
7:     self.obj.as_ref().map(|arc| {
8:         let new_cap = Capability {
9:             obj: Some(Arc::clone(arc)),
10:            flags: new_flags & self.flags,
11:            shared_to: Mutex::new(Vec::new()),
12:            original: false
13:        };
14:
15:        // Store weak reference to the new capability
16:        let new_cap_arc = Arc::new(new_cap.clone());
17:        self.shared_to.try_lock().unwrap()
18:            .push(Arc::downgrade(&new_cap_arc));
19:
20:        new_cap
21:    })
22: }
```

Der Share-Systemaufruf besteht aus drei Parametern: der zu teilenden Capability, dem Ziel-Thread sowie den zu übertragenden Rechten.

Zur Durchführung öffnet der Kernel den CSpace des aufrufenden Threads und erstellt einen Klon der angegebenen Capability. Die ursprüngliche Capability erhält dabei eine Referenz auf die neu erzeugte Capability (Quellcode 8). Diese Referenz wird benötigt, um im Fall einer späteren Revocation alle abgeleiteten Capabilities identifizieren zu können.

Die Rechte der neuen Capability werden den angeforderten Berechtigungen entsprechend eingeschränkt. Es gilt das Prinzip der monotonen Rechtevergabe: Es können ausschließlich Rechte weitergegeben werden, über die der aufrufende Thread selbst verfügt. Fordert der aufrufende Thread beispielsweise Lese- und Schreibrechte an, besitzt jedoch nur Leserechte, so wird die neue Capability ausschließlich mit Leserechten erzeugt.

Anschließend wird die erzeugte Capability im CSpace des Ziel-Threads abgelegt.

Der Empfänger muss über den Erhalt der Capability informiert werden. Aktuell kann dies über eine öffentliche Pipe erfolgen, auf die alle Prozesse zugreifen können. Dieser Ansatz ist jedoch unsicher, da Nachrichten von beliebigen Prozessen mitgelesen werden können. Für eine robuste Lösung ist daher ein privater Kommunikationsmechanismus erforderlich, der Vertraulichkeit und Integrität der Benachrichtigung gewährleistet (Siehe Kapitel 7.1).

Quellcode 9: Share Test-Anwendung (pipetest.rs)

```

1: pub fn main() {
2:     let res = mkfifo("mypipe",
3:         OpenOptions::all(),
4:         ROOT);
5:     if res.is_err() {
6:         return;
7:     }
8:     let pipe_cap = res.unwrap();
9:
10:    let Ok(open_pipe) = open(pipe_cap,
11:        OpenOptions::READWRITE)
12:    else {
13:        return;
14:    };
15:    let writer = thread::create(|| {
16:        writer_thread();
17:    });
18:
19:    if let Some(w) = writer {
20:        let num = share_naming_object(w.id(),
21:            OpenOptions::all(),
22:            pipe_cap);
23:        if num < 0 {
24:            return;
25:        }
26:
27:        println!("Shared pipe cap {} with writer thread {}",
28:            num, w.id());
29:        let buff= [num as u8];
30:        let res = write(SHARED_PIPE, &buff);
31:        w.join()
32:    }
33:    let reader = thread::create(|| {
34:        reader_thread();
35:    });
36:
37:    if let Some(r) = reader {
38:        let num = share_naming_object(r.id(),
39:            OpenOptions::all(),
40:            pipe_cap);
41:        let buff= [num as u8];
42:        let res = write(SHARED_PIPE, &buff);
43:        r.join();
44:    }
45: }

```

```

QEMU
p9OS v0.1.0 (debug) | Uptime: 00:01:39 | Processes: 2 | Threads: 3 | 2026-03-24 13:59:41
# Version      : v0.1.0 (debug - 00)
# Git Ref      : development - 738f814
# Build Date   : 2026-03-18 14:10:16
# Compiler     : rustc 1.91.0-nightly
# Bootloader   : towboot 0.9.5

> pipetest
named pipe demo: start, id 7
got root capability
mkfifo: ok, cap_handle = Capability { handle: 2 }
Read: Hello from main thread!
Starting writer, id 8
Shared pipe cap 3 with writer thread 8
Sent pipe cap to writer thread 8
---writer_thread: start, id 8
---writer_thread: got capability handle = Capability { handle: 3 }
---writer_thread: wrote one byte = 'A'
---writer_thread: wrote one byte = 'B'
---writer_thread: wrote one byte = 'C'
---writer_thread: wrote one byte = 'D'
---writer_thread: wrote one byte = 'E'
---writer_thread: wrote one byte = 'F'
---writer_thread: wrote one byte = 'G'
---writer_thread: end
Writer done, starting reader
+++reader_thread: read one byte 'A', read = 1
+++reader_thread: read one byte 'B', read = 1
+++reader_thread: read one byte 'C', read = 1
+++reader_thread: read one byte 'D', read = 1
+++reader_thread: read one byte 'E', read = 1
+++reader_thread: read one byte 'F', read = 1
+++reader_thread: read one byte 'G', read = 1
+++reader_thread: end
named pipe demo: done
>

```

Abbildung 10: Demo einer geteilten Pipe

Zur Demonstration des Share-Mechanismus wurde die Anwendung „pipetest“ (Quellcode 9) angepasst. Die Anwendung erstellt zunächst eine Pipe, verwendet diese jedoch nicht direkt. Stattdessen wird die zugehörige Capability an separate Lese- und Schreib-Threads delegiert. Diese greifen folglich nicht auf die ursprüngliche Capability zu, sondern ausschließlich auf die jeweils erhaltenen Klone.

Der Schreib-Thread schreibt asynchron die Buchstaben „A“ bis „G“ in die Pipe, während der Lese-Thread diese ausliest. Die Kommunikation funktioniert wie erwartet, siehe Abbildung 10.

5.6 Revocation



```

QEMU
v0S v0.1.0 (debug) | Uptime: 00:00:15 | Processes: 2 | Threads: 4 | 2026-03-24 14:02:41
# Version      : v0.1.0 (debug - 00)
# Git Ref      : development - 738f814
# Build Date   : 2026-03-18 14:10:16
# Compiler     : rustc 1.91.0-nightly
# Bootloader   : towboot 0.9.5

> revoketest
2nd Process Waiting for capability to be shared...
3rd Process Waiting for capability to be shared...
> Process 2: Successfully opened shared pipe
Shared Capability with 3rd process
Process 3: Successfully opened shared pipe
Process 2, waiting for capability to be revoked
Process 2 cap revoked
Process 3, waiting for capability to be revoked
Process 3 cap revoked

```

Abbildung 11: Demo einer entzogenen Naming Capability

Die Möglichkeit, Capabilities zu teilen, ist zentral für flexible Zugriffskontrolle. Ohne einen entsprechenden Mechanismus zum Entzug dieser Rechte würde die Kontrolle über vergebene Berechtigungen verloren gehen. Daher wird im Folgenden die Revocation von Capabilities betrachtet.

Um die Funktionsweise des Revocation-Systems zu überprüfen, wurde eine Testanwendung entwickelt. Dabei erstellt ein erster Prozess eine Pipe und teilt die zugehörige Capability mit einem zweiten Prozess. Dieser gibt die Capability anschließend an einen dritten Prozess weiter. Abschließend entzieht der erste Prozess die zuvor vergebenen Capabilities wieder (siehe [Abbildung 11](#) und [Quellcode 10](#)).

Wie bereits in [Abschnitt 3.1.1](#) beschrieben, muss eine Zugriffskontrolle in der Praxis den selektiven und partiellen Rechte-Entzug unterstützen. Eigenschaften wie transitive, unabhängige, direkte oder temporale Revokation sind dabei optional.

Quellcode 10: Testanwendung (revoketest.rs & revoketest1.rs)

```

1: fn revoke_thread() {
2:     let Ok(file) = mkfifo("revoke",
3:         OpenOptions::all(), R00T) else {
4:         return;
5:     };
6:
7:     //Share Capability
8:     share_naming_object(10,
9:         OpenOptions::all(),
10:        file);
11:
12:    // Revoke the capability
13:    revoke_naming_object(10, file);
14:    revoke_naming_object(12, file);
15: }
16:
17: pub fn main() {
18:     let pipe = Capability::new(2); //Shared Cap at index 2
19:     //...
20:
21:     //Share capability
22:     share_naming_object(12, OpenOptions::all(), pipe);
23:
24:     loop {
25:         let Ok(open_pipe) = open(pipe,
26:             OpenOptions::READWRITE) else {
27:             println!("Process 2 cap revoked");
28:             return;
29:         };
30:         if write(open_pipe
31:             , "Process 2: Capability revoked, write should fail"
32:             .as_ref())
33:             .is_ok(){
34:             println!("Process 2, waiting for revoke");
35:         } else {
36:             println!("Process 2 cap revoked")
37:         }
38:         close(open_pipe);
39:     }
40: }

```

5.6.1 Selektiv und Partiell

Die Revokes sind selektiv, da es möglich ist, unter Angabe der zu entziehenden Capability und der Nummer des betroffenen Threads die jeweilige Capability zu entziehen. Dies

ist für alle Threads, also auch für eine beliebige Teilmenge möglich. Jedoch unter der Einschränkung, dass der Ziel-Thread die Capability zuvor durch den ausführenden Thread oder einen seiner „Erben“ erhalten hat, also in der Vererbungshierarchie nachfolgt. Es ist hierbei auch möglich, nur einzelne Rechte zu entziehen, wobei von der Ziel-Capability einzelne Flags gelöscht werden. Somit ist die Eigenschaft „partiell“ auch erfüllt.

5.6.2 Transitiv

Transitive Revocation kann durch die Verwendung einer baumartigen Datenstruktur realisiert werden, welche sämtliche Ableitungen einer Capability erfasst [Lop18]. Im Falle eines Entzugs kann dieser Baum traversiert werden, um alle daraus hervorgegangenen Capabilities ebenfalls zu invalidieren.

Im Rahmen einer Implementierung mit Rust ergibt sich jedoch ein grundlegendes Problem im Zusammenhang mit dem Ownership-Modell. Die Verwendung von Arc erlaubt lediglich schwache Referenzen (Weak), über die zwar die Struktur der Ableitungen nachvollzogen werden kann, nicht jedoch ein direkter Zugriff zur Revokation möglich ist. Eine alternative Strategie bestünde darin, alle CSpaces zu durchsuchen und zu prüfen, ob eine Capability transitiv aus einer entzogenen Berechtigung hervorgegangen ist. Dieser Ansatz ist jedoch mit erheblichen Kosten verbunden, da die Laufzeit sowohl mit der Anzahl der Threads als auch mit der Anzahl der delegierten Capabilities skaliert.

Das betrachtete System unterstützt in seiner aktuellen Form keine effiziente transitive Revocation. Insbesondere in einem verteilten System wie D3OS es werden soll, würde ein solcher Ansatz zu erheblichen Performanceeinbußen führen, da globale Zustandsinformationen erforderlich wären und Revokationsoperationen nicht mehr lokal begrenzt durchgeführt werden könnten.

Ein weiteres Argument gegen transitive Revocation ergibt sich aus einer anwendungsbezogenen Perspektive. In der Praxis sind die meisten Anwendungen, denen Berechtigungen erteilt werden, nicht bösartig [Kar25]. Der Entzug einer Capability bei einer Anwendung impliziert daher nicht zwangsläufig, dass alle nachgelagerten Anwendungen diese Berechtigung ebenfalls verlieren sollten. Insbesondere können diese weiterhin legitime und funktional notwendige Verwendungen der Capability haben. Vor diesem Hintergrund erscheint eine selektive, nicht-transitive Revocation als die zweckmäßigere Strategie.

5.6.3 Independent

Hat ein Prozess von mehreren anderen Prozessen dieselbe Berechtigung erhalten (z.B. Lesen einer bestimmten Datei), so ist es in der aktuellen Implementierung nicht möglich, nur eine dieser Berechtigungen zu entziehen. Dies ist so beabsichtigt.

Im Capability-Modell ist es vorgesehen, nur Capabilities zu teilen, falls ein Prozess diese

unbedingt für seine Funktion benötigt. Ein Revoke sollte also nicht vorgesehen sein.

Gibt es nun jedoch trotzdem einen Grund, eine Capability zu entziehen, so kann angenommen werden, dass dieser so gravierend ist, dass ohne ein Revoke die Systemsicherheit bedroht ist. Ein Prozess der mehrere Berechtigungen auf das selbe Objekt besitzt, sollte also keine dieser Rechte behalten, falls er das System bedroht.

Demnach ist ein unabhängiger Entzug unter Umständen ein Sicherheitsrisiko. Ein Nachteil ist, dass „böartige“ Prozesse in der Praxis Rechte mit anderen Prozessen teilen könnten, um diese Berechtigung direkt zu entziehen und so von „gutartigen“ Prozessen deren rechtmäßige Berechtigungen zu entfernen.

Doch erhalten diese böartigen Prozesse überhaupt die Berechtigungen, so sind bereits Fehler im Vergabe-Prozess passiert. Außerdem ist ein erneutes Rechte-Erteilen zwar aufwendig, jedoch ohne Probleme wieder möglich, da dem ursprünglichen Rechteinhaber diese nicht entzogen werden können.

5.6.4 Immediate

Die Revocation erfolgt im aktuellen System unmittelbar. Sobald der entsprechende Systemaufruf verarbeitet wird, wird die betroffene Capability invalidiert, bevor ein weiterer Zugriff erfolgen kann. Es entstehen keine inkonsistenten Zustände, da ausschließlich in der Zukunft liegende Zugriffe unterbunden werden.

Aufgrund der Single-Core-Architektur werden laufende Operationen stets abgeschlossen, bevor ein neuer Systemaufruf verarbeitet wird. Das bedeutet, dass ein Zugriff, der bereits initiiert wurde, noch vollständig ausgeführt wird, selbst wenn unmittelbar danach eine Revocation erfolgt. Da diese Revocation erst im Anschluss verarbeitet werden kann.

In einem Multicore-System würde sich ein ähnliches Verhalten ergeben: Bereits gestartete Operationen, wie beispielsweise Lese- oder Schreibzugriffe, würden zu Ende ausgeführt werden, auch wenn in der Zwischenzeit ein Revoke-Befehl, etwa von einem anderen Kern verarbeitet wird. Es könnte durch diese Nebenläufigkeit nicht mehr von einem unmittelbaren Revoke gesprochen werden.

Ein Vorteil ist, dass dieses Modell nicht zu inkonsistenten Zuständen führt, da die Revocation ausschließlich zukünftige Zugriffe verhindert und nicht in laufende Operationen eingreift.

5.6.5 Temporal

Ein temporärer Rechteentzug dient dazu, das Verhalten eines Prozesses mit eingeschränkten Berechtigungen zu simulieren.

Wie in Abschnitt 5.2.1 beschrieben, kann dies alternativ durch die Ausführung in einem

separaten Thread mit reduzierten Rechten erreicht werden. Dieser Ansatz bietet eine vergleichbare Funktionalität.

Ein temporales Agieren mit weniger Berechtigungen ist zu selten notwendig und gewährt auch hier keine Isolation, welche ein eigener, separierter Prozess gewährleisten könnte. Es ist folglich nicht implementiert worden. Auch da Möglichkeiten diese Funktion nachzuahmen existieren, sollte es eine Anwendung benötigen.

6 Capabilities im Mikrokernel

In einem Mikrokernel Dateisystem übernimmt der Kernel nur wenige Aufgaben: Inter-Prozess-Kommunikation (IPC), Scheduling und Virtualisierung [Wik25b].

Alle anderen Aufgaben, die der Kernel in einem monolithischen Betriebssystem übernimmt, werden bei einem Mikrokernel-OS durch Treiber durchgeführt, welche im User-Mode laufen.

Hierbei sollte dennoch dieselbe Sicherheit gewährleistet sein, die Capability-basierte Zugriffskontrolle im Monolith-OS liefert.

6.1 Theorie

Im monolithischen System folgen Zugriffe immer einem gleichen Muster. Ein Nutzer möchte auf eine Datei, ein Gerät oder eine beliebige andere Ressource zugreifen. Er sendet den Befehl an das Betriebssystem - also den Kernel - dies durchzuführen. Hierfür legt er die Capability vor, die er vorher erhalten hat. Das Betriebssystem überprüft die Validität der Capability und gewährt oder verwehrt den Zugriff entsprechend.

Ein Treiber, welcher im Mikrokernel-OS im User-Mode ausgeführt wird, kann diese Validierung nicht durchführen. Er muss so aufgebaut sein, dass er auf den Kernel vertrauen kann, diese Validierung an seiner Statt auszuführen. Hierfür wird die Interprozesskommunikation verwendet:

Möchte ein Nutzer Befehle an einen Treiber senden, so muss er sich zunächst an den Kernel wenden, um einen IPC-Kanal zu dem Treiber zu erhalten. Vom Kernel erhält er die Capability, auf dem jeweiligen Kanal zu kommunizieren. Bei jedem Zugriff überprüft der Kernel die Berechtigungen.

Der Treiber erhält ebenfalls die Capability, um auf den Kanal zuzugreifen. Er kann darauf vertrauen, dass auf dem Kanal nur Nutzer schreiben oder lesen, die die Berechtigung zu diesem haben. Berechtigungen, wie etwa Dateiberechtigungen in einem Datei-Treiber, werden vom Treiber pro Kanal gewährt.

Zum Beispiel kann Kanal 1 eine Datei erstellen und erhält vollen Zugriff auf diese. Möchte Kanal 2 ebenfalls auf diese Datei zugreifen, so muss entweder die Genehmigung von Kanal 1 erteilt werden, oder der Nutzer am anderen Ende von Kanal 2 schreibt direkt mit über Kanal 1, indem er die Berechtigung zu dem Kanal erteilt bekommt.

Dem Treiber ist unbekannt, wer genau der Teilnehmer am anderen Ende ist, ob es nur ein Nutzer ist oder eventuell sogar mehrere. Dies zu wissen ist auch nicht notwendig. Es ist nur wichtig zu kennen, von welchem Kanal welche Befehle übermittelt werden.

6.2 Treiber-interne Rechteverwaltung

Obwohl der Kernel im Mikrokern-System sicherstellt, dass nur autorisierte Prozesse mit einem bestimmten Kommunikationskanal interagieren können, liegt die semantische Zugriffskontrolle auf konkrete Ressourcen vollständig in der Verantwortung des Treibers. Der Kernel überprüft lediglich die Gültigkeit von Capabilities für IPC-Endpunkte, besitzt jedoch kein Wissen über die Bedeutung der ausgetauschten Nachrichten oder die Struktur der vom Treiber verwalteten Objekte [Hei20].

Daher ist es notwendig, dass Treiber eigene Mechanismen zur Absicherung interner Ressourcen implementieren. Dies betrifft insbesondere Objekte wie Dateien, Handles oder andere abstrakte Ressourcen, die durch den Treiber bereitgestellt werden. Zwei grundlegende Ansätze sind hierbei üblich.

In der vorliegenden Implementierung wird ein vereinfachtes Modell verwendet. Jeder Server-Thread verwaltet einen eigenen isolierten Zustandsraum, wodurch jeder Kommunikationskanal effektiv einer separaten Sicherheitsdomäne entspricht. Die vom Treiber vergebenen Handles sind ausschließlich innerhalb dieses Kontexts gültig und können nicht kanalübergreifend verwendet werden. Eine Weitergabe von Rechten zwischen verschiedenen Clients ist daher nur möglich, indem der Zugriff auf den entsprechenden Kommunikationskanal explizit geteilt wird.

Die hier vorgestellte Implementierung dient ausschließlich Demonstrationszwecken. Eine reale Datei- oder Speicherverwaltung erfordert deutlich umfangreichere Mechanismen zur Zugriffskontrolle. Insbesondere sind fein granular definierte Berechtigungen, die Trennung von Identität und Kommunikationskanal, sowie Mechanismen zur Delegation und zum Entzug von Rechten notwendig. Darüber hinaus müssen Aspekte wie Persistenz, gleichzeitiger Zugriff mehrerer Clients, sowie konsistente Zustandsverwaltung berücksichtigt werden.

6.3 Demonstration

Da im bestehenden System noch keine User-Mode-Treiber implementiert sind, wird zur Demonstration der durch die Zugriffskontrolle abgesicherten Kommunikation ein vereinfachter Dateisystem-Treiber entwickelt.

6.3.1 Treiber

Quellcode 11: Start des Dateitreibers (fileserver.rs)

```
1: pub fn main() -> () {  
2:     // Create the monitoring thread  
3:     let _ = create_monitor_thread();  
4: }
```

Beim Start über das Terminal wird ein Monitor-Thread erzeugt (Quellcode 11). Dieser schreibt seine Thread-ID in die öffentliche Pipe und wartet darauf, dass ihm eine Capability zugewiesen wird. Sobald dies geschieht, startet er einen neuen Thread, der eingehende Anfragen über die durch die Capability referenzierte Pipe verarbeitet (Quellcode 12).

Quellcode 12: Warten bis eine Capability geteilt wird (filesrvr.rs)

```

1: fn monitor_thread() {
2:     let id = thread::current().unwrap().id() as u8;
3:
4:     let naming_len = 2;
5:     let Ok(open_shared_pipe) =
6:         open(SHARED_PIPE, OpenOptions::READWRITE) else {
7:             println!("Failed to open shared pipe in monitor thread");
8:             return;
9:         };
10:
11:     for i in 0..10 { //do more if I want to connect more clients
12:         write(open_shared_pipe, &[id]);
13:     }
14:
15:     loop { //Warten bis Capability geteilt wird
16:         if capabilities::get_naming_len() > naming_len {
17:             //create a server thread
18:             let _ = create(server_thread1);
19:         }
20:     }
21: }

```

Der erzeugte Server-Thread verarbeitet einfache Befehle eines Clients. Für Demonstrationszwecke unterstützt der Treiber lediglich zwei Operationen: das Schreiben eines Zeichens in eine Datei sowie das anschließende Auslesen dieses. Anstelle eines einzelnen Zeichens könnte in einer realen Implementierung der vollständige Dateinhalt übertragen werden.

Jeder Server-Thread verwaltet dabei einen eigenen isolierten Dateibereich. Dadurch wird sichergestellt, dass nur der jeweilige Client Zugriff auf seine Daten erhält. In dieser vereinfachten Implementierung erfolgt die Speicherung ausschließlich im Arbeitsspeicher, wodurch alle Daten bei einem Neustart verloren gehen. Ein produktiver Treiber würde stattdessen persistente Speichermechanismen verwenden.

Quellcode 13: Anfrageverarbeitung (fileservr.rs)

```

1: pub(crate) fn run(&mut self) -> Result<(), Errno> {
2:     let mut cmd_buf = [0u8; 2]; // 1 byte command + 8 bytes data
3:
4:     //...
5:
6:     loop {
7:         // Read command from pipe
8:         let _ = read(openpipe, &mut cmd_buf);
9:         println!("File server received command: {}", cmd_buf[0]);
10:
11:         match cmd_buf[0] {
12:             // Write command
13:             1 => {
14:                 let letter:u8 = cmd_buf[1].clone();
15:                 let handle = self.next_handle;
16:                 self.next_handle += 1;
17:                 self.files.insert(handle, letter.clone());
18:
19:                 let response = handle;
20:                 write(openpipe, &[response as u8])?;
21:             }
22:
23:             // Read command
24:             2 => { // Read
25:                 let handle = cmd_buf[1].clone() as usize;
26:                 if let Some(&letter) = self.files.get(&handle) {
27:                     write(openpipe, &[letter as u8])?;
28:                 } else {
29:                     write(openpipe, &[0])?;
30:                 }
31:                 break; //enough for this demo
32:             }
33:
34:             _ => {} // Invalid command
35:         }
36:     }
37:     Ok(())
38: }

```

6.3.2 Nutzer

Quellcode 14: Einstieg der Nutzeranwendung (fileclient.rs)

```
1: pub fn main(){
2:     let Some(mut client) = FileClient::connect() else {
3:         print!("fileclient: failed to connect to file server\n");
4:         return;
5:     };
6:
7:     let handle = client.write_file(b'A')
8:         .expect("Failed to write letter");
9:     print!("Wrote 'A' with handle: {}\n", handle);
10:
11:     // Read it back
12:     let Ok(letter) = client.read_file(handle) else {
13:         print!("Failed to read file with handle: {}\n", handle);
14:         return;
15:     };
16:     print!("Read back: {}\n", letter as char);
17:
18: }
```

Die Nutzeranwendungen übernehmen in dieser Demonstration unterschiedliche Rollen. Gemeinsam ist ihnen lediglich der initiale Verbindungsaufbau zum Treiber, sofern sie einen eigenen Kommunikationskanal verwenden.

Anwendung 1 baut eine eigene Verbindung zum Treiber auf und nutzt diesen Kanal exklusiv. Sie schreibt den Buchstaben „A“ in eine Datei und erhält dafür ein Handle, das innerhalb ihres Server-Kontexts gültig ist. Anschließend liest die Anwendung den Wert über denselben Kanal wieder aus. Dieser Ablauf entspricht einem vollständig isolierten Zugriff ohne Interaktion mit anderen Anwendungen.

Anwendung 2 baut ebenfalls eine eigene Verbindung zum Treiber auf und erzeugt damit einen separaten Server-Thread mit eigenem Zustandsraum. Sie schreibt den Buchstaben „B“ und erhält ein entsprechendes Handle. Im Gegensatz zu Anwendung 1 nutzt Anwendung 2 den Kanal danach jedoch nicht weiter selbst, sondern delegiert den Zugriff. Hierzu teilt sie die Capability ihrer privaten Pipe mit Anwendung 3.

Anwendung 3 stellt keine eigene Verbindung zum Treiber her. Stattdessen verwendet sie ausschließlich die von Anwendung 2 erhaltene Capability. Über diesen bereits etablierten Kommunikationskanal kann Anwendung 3 direkt mit dem zugehörigen Server-Thread interagieren. Sie sendet einen Lesebefehl unter Verwendung des von Anwendung 2 erzeugten Handles und erhält den gespeicherten Wert „B“.

Quellcode 15: Aufbauen der Verbindung (fileclient.rs)

```

1: pub fn connect() -> Option<Self> {
2:     let Ok(pipe) = mkfifo("client1",
3:         OpenOptions::all(),
4:         ROOT) else {
5:         return None;
6:     };
7:
8:     let mut thread_id = [1u8];
9:     let mut ack = [0u8; 1];
10:
11:     let Ok(opensharedpipe) = open(SHARED_PIPE,
12:         OpenOptions::READWRITE) else {
13:         return None;
14:     };
15:
16:     //read server thread id from shared pipe
17:     read(opensharedpipe, &mut thread_id);
18:
19:     //share client pipe with server
20:     share_naming_object(
21:         thread_id[0].clone() as usize,
22:         OpenOptions::all(), pipe);
23:
24:     let Ok(openpipe) = open(pipe,
25:         OpenOptions::READWRITE) else {
26:         return None;
27:     };
28:
29:     read(openpipe, &mut ack); //read ACK
30:
31:     Some(FileClient { pipe_cap: pipe })
32: }

```

Der Verbindungsaufbau (Quellcode 15) erfolgt über die öffentliche Pipe. Die Anwendung liest zunächst die Thread-ID des Treibers aus dieser Pipe. Anschließend erstellt sie eine eigene private Pipe und teilt die entsprechende Capability mit dem Treiber-Thread. Sobald über diese Pipe eine Bestätigung empfangen wird, ist die Verbindung hergestellt.

Die Kommunikation mit dem Treiber erfolgt ausschließlich über die jeweilige private Pipe. Befehle werden in den Kanal geschrieben und die entsprechenden Antworten daraus gelesen.

Insgesamt ergibt sich folgendes Ablaufverhalten: Der Dateitreiber sowie drei Nutzeranwendungen werden gestartet. Anwendung 1 und Anwendung 2 bauen jeweils eigene Verbindungen zum Treiber auf und erhalten getrennte Server-Threads mit isolierten Zustandsräumen. Anwendung 1 arbeitet vollständig unabhängig und liest den zuvor geschriebenen Wert „A“ wieder aus ihrem eigenen Kontext.

Anwendung 2 erzeugt ein Objekt im Treiber und delegiert anschließend den Zugriff durch Weitergabe ihrer Capability an Anwendung 3. Anwendung 3 nutzt diesen bestehenden Kommunikationskanal und greift auf denselben Server-Kontext zu. Dadurch kann sie den von Anwendung 2 gespeicherten Wert „B“ auslesen.

Siehe Abbildung 12.

Dies zeigt, dass die Zugriffskontrolle vollständig an den Besitz einer Capability gebunden ist. Während getrennte Kanäle zu Isolation führen, ermöglicht die Weitergabe einer Capability eine explizite Delegation von Zugriffsrechten.

Quellcode 16: Befehle an den Dateitreiber (fileclient.rs)

```

1:  pub fn write_file(&self, letter: u8) -> Result<FileHandle, Errno> {
2:      let Ok(openpipe) = open(self.pipe_cap,
3:          OpenOptions::READWRITE) else {
4:          return Err(Errno::ENOENT);
5:      };
6:
7:      // Send write command (1) and the letter
8:      write(openpipe, &[1, letter])?;
9:
10:     // Read back handle
11:     let mut handle_buf = [0u8; 1];
12:
13:     read(openpipe, &mut handle_buf)?;
14:
15:     Ok(handle_buf[0].clone() as usize)
16: }
17:
18: pub fn read_file(&self, handle: FileHandle) -> Result<u8, Errno> {
19:     let Ok(openpipe) = open(self.pipe_cap,
20:         OpenOptions::READWRITE) else {
21:         return Err(Errno::ENOENT);
22:     };
23:
24:     // Send read command (2) and the handle
25:     write(openpipe, &[2, handle as u8])?;
26:
27:     // Read back the letter
28:     let mut letter = [5u8];
29:     read(openpipe, &mut letter)?;
30:
31:     Ok(letter[0].clone())
32: }

```

```

QEMU
vPOS v0.1.0 (debug) | Uptime: 00:01:34 | Processes: 3 | Threads: 3 | 2026-03-24 14:02:00
# Version      : v0.1.0 (debug - 00)
# Git Ref       : development - 738f814
# Build Date    : 2026-03-18 14:10:16
# Compiler      : rustc 1.91.0-nightly
# Bootloader    : towboot 0.9.5

> fileserver
---fileclient: connecting to file server...
---fileclient: created client pipe with cap = 2
---fileclient: got thread id = 8
---fileclient2: connecting to file server...
---fileclient2: created client pipe with cap = 2
---fileclient2: got thread id = 8
Monitor thread: Detected new client, started server thread
File server thread started for pipe at 2
sending ack to client
File server waiting for command...
---fileclient: got server response = 1. Client Connected!
File server received command: 1
File server stored letter 'A' with handle 1
Wrote 'A' with handle: 1
File server received command: 2
File server sent letter '65' for handle 1
Read back: A
Monitor thread: Detected new client, started server thread
File server thread started for pipe at 3
sending ack to client
File server waiting for command...
---fileclient2: got server response = 1. Client Connected!
File server received command: 1
File server stored letter 'B' with handle 1
Wrote 'B' with handle: 1
File server received command: 2
File server sent letter '66' for handle 1
Read back: B

```

Abbildung 12: Demo des Dateitreibers

7 Zusammenfassung und Ausblick

Im Laufe dieser Arbeit wurde eine Capability-basierte Zugriffskontrolle entwickelt.

Diese deckt die volle Zuständigkeit eines Mikrokernels ab: Die abgesicherten Systemcalls stellen sicher, dass Anwendungen nur diejenigen Befehle ausführen können, zu denen sie berechtigt wurden, wodurch alle Kernel-Ressourcen, insbesondere die Speicherverwaltung und das Scheduling abgesichert sind. Die Überarbeitung des Naming-Services sichert dabei die aktuell einzige Möglichkeit für Inter-Prozess-Kommunikation.

Dabei erfüllt das System alle Kriterien einer Zugriffskontrolle hinreichend:

1. Confidentiality: Prozesse erhalten keine Informationen über existierende Naming-Objekte. Einschränkung: Um eine Anwendungsentwicklung zu ermöglichen, steht die Syscall-Reihenfolge fest. Die Existenz der Syscalls bleibt hier somit derzeit bekannt.
2. Integrity: Ohne die benötigten Capability-Objekte ist es einem Prozess nicht möglich auf (Naming-) Ressourcen zuzugreifen. Da Zugriffe nur über den prozesseigenen CSpace verlaufen, können Prozesse nicht auf fremde, ihnen nicht zugewiesene Ressourcen zugreifen.
3. Availability: Systemressourcen, die zuvor zur Verfügung standen, sind weiterhin verfügbar. Lediglich die Adressierung der Ressourcen nicht aber deren Verfügbarkeit wurde verändert.
4. Integrity: Die Erteilung von Berechtigungen ist über die Adressierung durch Thread-IDs möglich. Berechtigungen können zwischen Nutzern geteilt und entzogen werden.

Durch den expliziten Capability Support des Naming-Systems, ist außerdem vorbereitet, dass eine zukünftige Verschiebung von Funktionen des Betriebssystems in User-Mode Treibern, die grundlegende Sicherheit der Zugriffskontrolle über die Interprozesskommunikation weiterverwenden kann. Dies wurde anhand eines simplen Beispiel-Treibers demonstriert.

Sollte die Auslagerung einzelner Funktionen entgegen der Mikrokern-Philosophie nicht erwünscht sein, können Capabilities entsprechend erweitert werden, um auch diese Funktionen in die Zugriffskontrolle zu integrieren.

Bei der Entwicklung dabei auf die Einhaltung der Designkriterien 1.3 nach Saltzer geachtet: Das Prinzip des Least Privilege wird durch die Capability-basierte Vergabe von Zugriffsrechten konsequent umgesetzt, da Prozesse ausschließlich über explizit zugewiesene Berechtigungen verfügen, die nach Open Design nicht auf Passwort-Sicherheit basieren. Auch Fail-Safe Defaults sind gegeben, da ohne entsprechende Capability grundsätzlich kein Zugriff möglich ist. Durch die zentrale Rolle des CSpace wird außerdem eine weitgehende Complete Mediation erreicht, da alle Zugriffe kontrolliert über diesen Mechanismus erfolgen, während für Nutzer möglichst nur geringe Änderungen vorliegen, ganz nach Psychological Acceptability.

Einschränkungen zeigen sich aktuell insbesondere beim Least-Common Mechanism, da die Verwendung einer öffentlichen Pipe einen geteilten Kommunikationskanal darstellt. Weitere Prinzipien wie die Separation of Privilege wurden in dieser Arbeit nicht vollständig adressiert und bieten Ansatzpunkte für zukünftige Erweiterungen.

Für die weitere Entwicklung in der Zukunft bleiben derzeit die folgenden Themen offen. Dies ist unter anderem dem geschuldet, dass die jeweils notwendigen Architekturen in dem aktuellen Betriebssystem-Stand nicht implementiert sind und eine Implementierung außerhalb des Rahmens dieser Arbeit liegen.

7.1 Messaging

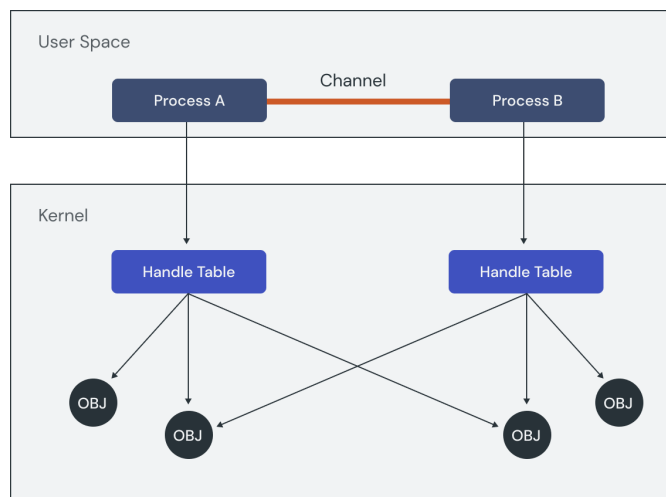


Abbildung 13: Channel-Architektur in Google-Fuchsia[Goo25]

Wie zuvor beschrieben wird derzeit eine öffentliche Pipe verwendet, auf die alle Prozesse zugreifen können. Hierüber wird zum Beispiel übermittelt, wenn ein Prozess eine neue Capability erhalten hat und welche Nummer diese erhält. Dies ist sehr unsicher. Solche privaten Nachrichten sollten über ein adäquateres Medium versendet werden. Hierbei wäre eine Lösung Message-Passing [IOA⁺21b]. Dabei erhält jeder Prozess einen Endpunkt, der ähnlich eines Briefkastens agiert. Nur der jeweilige Prozess bekommt die Lese-Berechtigung. Schreibberechtigungen können delegiert werden.

Die Rechteverwaltung läuft auch hier über Capabilities [KAE⁺14].

Der wichtige Unterschied besteht darin, dass jeder Prozess aktuell nur die Möglichkeit hat, eine Pipe zu erstellen und diese mit einem anderen Prozess zu teilen, um miteinander zu kommunizieren. Der zweite Prozess muss hierbei etwa über die öffentliche Pipe informiert

werden, dass er eine private Pipe „erhalten“ hat. Beim Message-Passing besitzt jeder Prozess bereits einen Endpunkt, von dem nur er selbst lesen kann. Für die Kommunikation müssen also lediglich Nachrichten versendet werden, anstelle zuvor die Kommunikation aufzubauen.

7.2 Multicore

Die aktuelle Implementierung von D3OS unterstützt nur Single-Core Operationen. Prinzipiell funktioniert das Capability-System auch unter Multi-Core Ausführung.

Sinnvoll wäre es, die Änderungen an den CSpaces und somit an den Berechtigungen der Prozesse transaktionell durchzuführen. Auf diese Weise bleiben Veränderungen konsistent, falls mehrere Threads oder vielmehr Prozessorkerne zeitgleich Änderungen durchführen wollen.

7.3 Restartability

Derzeit wird im Rahmen von D3OS an der Neustartbarkeit des Betriebssystems nach Stromausfällen, Shutdowns und Fehlern geforscht. Dabei wird „Checkpointing“ verwendet, um regelmäßig die Zustände der Prozesse zu sichern. Es gehen also immer kurze Zeiträume verloren, jedoch kann der Großteil der Informationen wiederhergestellt werden.

Aktuell wird der Kernel des Betriebssystems nicht mit gesichert. Dieser verliert also den Zustand vor dem Neustart. Da die Capabilities in der beschriebenen Implementierung nicht beim User, sondern im Kernel liegen, gehen sämtliche Berechtigungen verloren. Um diese im Laufe der Wiederherstellung auch wiederzuerteilen, sollte es einen Weg geben, Capability-Zustände zu sichern, etwa indem sie bei jeder Änderung serialisiert und auf der Festplatte gespeichert werden. Eine Möglichkeit der Serialisierung wäre es, die Capability Objekte in JSON oder anderen bekannten Dateiformaten zu sichern. Hierfür könnten eventuell existierende Frameworks hinzugezogen werden. Bei der Wiederzuweisung kann dadurch der CSpace eines Prozesses anhand der gesicherten Capabilities befüllt werden. Dabei ist darauf zu achten, dass die Berechtigungen auf der Festplatte nicht manipuliert werden können. Auch muss eine bessere Identifikation der Prozesse angeboten werden. Die aktuellen inkrementelle Prozess-ID kann nur verwendet werden, wenn Prozesse exakt in derselben Reihenfolge neu gestartet werden. Hier könnte eine genauere Identifikation nützlich sein.

Eine andere Methode wäre, eine Notation zu entwickeln, um weniger Daten sichern zu müssen und so das Checkpointing zu beschleunigen. CapDL[[KKLW10](#)] wurde für diesen Zweck speziell für seL4 entwickelt. Berechtigungen werden Zeile für Zeile gesichert, anstelle von ganzen JSON-Objekten. Um etwa einem Prozess Lese- und Schreib-Berechtigungen zu einem Treiber zu gewähren und ihm dabei wieder eine Identifikation zuzuteilen, kann dies so verfasst werden:

```
CLIENT1_cspace = { 1: DRIVER_aep (RW, badge: 1) }
```

Da die Capability Implementierung in seL4 leicht von der Implementierung in D3OS abweicht, könnte hier die genaue Notation angepasst werden. Alternativ wäre mehr Aufwand in der Übersetzung nötig.

7.4 Shared-Memory

Um Shared Memory mit Capabilities abzusichern, werden virtuelle Adressen verwendet [HRK91]. Dabei wird ein gemeinsam genutzter Speicherbereich als eigenständiges Objekt modelliert, auf das ausschließlich über eine zugehörige Capability zugegriffen werden kann. Eine Capability erhält die notwendigen Metadaten, um ein Speichersegment eindeutig zu referenzieren und den Zugriff darauf zu kontrollieren. Typischerweise speichert sie den Namen oder Identifikator des Segments sowie dessen Länge. Zusätzlich enthält sie die zulässigen Zugriffsrechte. Der Zugriff auf den gemeinsam genutzten Speicher erfolgt nicht direkt

Segment Capability

Segment Name	Length	Access Info
--------------	--------	-------------

Abbildung 14: Speichersegment Capability [HRK91]

über rohe Speicheradressen, sondern ausschließlich über die Capability. Wird ein Zugriff ausgeführt, werden zunächst die in der Capability gespeicherten Informationen ausgewertet. Anschließend erfolgt die Auflösung der virtuellen Adresse über die Seitentabellen des Shared-Memory-Managements. Diese übersetzen die virtuelle Adresse in die entsprechende physische Speicheradresse. Damit übernimmt die Capability die Rolle der Zugriffskontrolle auf das Speichersegment, während die eigentliche Adressübersetzung weiterhin durch das virtuelle Speicher- und Paging-System des Betriebssystems erfolgt. Auf diese Weise wird der Zugriff auf Shared Memory sowohl durch die Berechtigungen der Capability als auch durch die Mechanismen der Speicherverwaltung kontrolliert.

Ein ähnlicher Ansatz könnte auch für das bestehende System implementiert werden. Da jedoch die aktuelle Speicherverwaltung kein Shared-Memory unterstützt, fehlt die Voraussetzung, um mehrere Akteure gleichzeitig auf identische Speicherbereiche zugreifen zu lassen.

In der Konsequenz würde ein Teilen von Berechtigungen ins Leere laufen. Selbst wenn man Zugriffsrechte auf Teilbereiche definiert, könnten diese faktisch nur dem jeweiligen Besitzer zugewiesen werden. Eine echte Erweiterung des Zugriffskonzepts oder ein funktionaler Mehrwert entsteht dadurch nicht.

7.5 Zusammenfassung

Zusammenfassend demonstriert die implementierte Capability-basierte Zugriffskontrolle eine sichere und feingranulare Verwaltung von Ressourcen in D3OS, wobei Benchmarks (C) eine vertretbare Performanceeinbuße von etwa 25 % aufzeigen.

Alle Sicherheitsziele einer Zugriffskontrolle nach Stallings werden dabei vollständig erfüllt. Insbesondere wird durch den im Kernel verankerten CSpace eine zentrale Instanz geschaffen, die analog zu seL4 eine Capability-basierte, passwortfreie und offen gestaltete Sicherheitsarchitektur ermöglicht.

Dabei wurden Dateizugriffe, angelehnt an EROS, explizit über Capabilities abgesichert. Dadurch wird sichergestellt, dass auch der aktuelle IPC-Mechanismus durch den Kernel kontrolliert wird und grundlegende Anforderungen moderner Mikrokern-Architekturen erfüllt. Zukünftige IPC-Mechanismen können auf diesem Ansatz aufbauen und nahtlos in die bestehende Zugriffskontrolle integriert werden.

Auch die Verwaltung und Weitergabe von Capabilities erfüllt zentrale Anforderungen. Nach dem Vorbild von Fuchsia wurde im Share-Mechanismus eine baumartige Struktur implementiert, die eine nachvollziehbare Delegation von Rechten ermöglicht und zyklische Abhängigkeiten verhindert. Diese Struktur bildet die Grundlage für das Revoke-System, welches einen konsistenten und sicheren Entzug von Berechtigungen gewährleistet.

Insgesamt schafft die entwickelte Capability-basierte Zugriffskontrolle damit eine belastbare Grundlage für zukünftige Erweiterungen. Insbesondere in den Bereichen Messaging, Multicore-Unterstützung, persistente Berechtigungen und Shared-Memory-Management.

Somit stellt die Arbeit einen funktionsfähigen und übertragbaren Ansatz für die Integration Capability-basierter Zugriffskontrollen in Mikrokern-Systemen dar und zeigt, dass sich die angestrebten Sicherheits- und Strukturprinzipien praktisch und konsistent umsetzen lassen.

Anhang

A Quellcode

Der vollständige Quellcode der Capability-Implementierung ist im GitLab-Repository verfügbar unter <https://git.hhu.de/bsinfo/thesis/ma-jusch322> [Sch26b] und im Github-Repository unter <https://github.com/Schn0ck/D30S-Capabilities> [Sch26a].

B Unterstützende Nutzung generativer KI-Werkzeuge

Für diese Arbeit wurden generative KI-Werkzeuge als unterstützende Hilfsmittel eingesetzt.

GitHub Copilot [Ope25b] und Claude Sonnet [Ant25] kamen zur Codevervollständigung, Optimierung und Fehleranalyse zum Einsatz. ChatGPT [Ope25a] wurde zur sprachlichen Überarbeitung und Verbesserung der Formulierungen verwendet.

Alle von den KI-Werkzeugen vorgeschlagenen Inhalte wurden sorgfältig geprüft, angepasst und getestet. Entscheidungen über wissenschaftliche Argumentation, inhaltliche Strukturierung und die Auswahl der Methoden lagen vollständig beim Verfasser.

C Benchmarks

Zur Abschätzung des Einflusses der Zugriffskontrolle auf die Zugriffszeit wurde ein Benchmark sowohl auf dem vorherigen als auch auf dem aktuellen Systemstand durchgeführt. Hierzu wurde eine Datei `/Test` im Root-Verzeichnis erstellt und anschließend 10.000 Mal der Buchstabe `A` in diese Datei geschrieben.

Im System ohne Zugriffskontrolle betrug die Gesamtlaufzeit der Schreibzugriffe 241 ms, im System mit Zugriffskontrolle 305 ms. Daraus ergibt sich eine mittlere Zugriffszeit von 24,1 μ s bzw. 30,5 μ s pro Schreiboperation. Die Zugriffskontrolle erhöht die durchschnittliche Zugriffszeit somit um 6,4 μ s pro Operation, was einer relativen Steigerung von etwa 26,5 % entspricht.

Zusammenfassend zeigen die durchgeführten Messungen, dass die Implementierung der Zugriffskontrolle mit einem messbaren Performance-Overhead verbunden ist. Der resultierende Mehraufwand ist damit klar quantifizierbar, bleibt jedoch im Bereich weniger Mikrosekunden pro Zugriff. Im Gegenzug wird durch die Zugriffskontrolle eine zusätzliche Sicherheitsschicht eingeführt, die unautorisierte Zugriffe verhindert und somit die Integrität des Systems stärkt. Insgesamt ergibt sich ein klassischer Konflikt zwischen Performance und Systemsicherheit.

Literatur

- [Ant25] ANTHROPIC: *Claude Sonnet 3.5*. <https://claude.ai/>. Version: 2025. – Large Language Model zur Unterstützung bei Code-Optimierung und Fehlersuche in Rust.
- [Bar97] BARKLEY, John: Comparing simple role based access control models and access control lists. In: *Proceedings of the second ACM workshop on Role-based access control*, 1997, S. 127–132
- [Bet24a] BETRIEBSSYSTEME, Heinrich-Heine-Universität: L.: *D3OS*. <https://github.com/hhu-bsinfo/D3OS>. Version: 2024
- [Bet24b] BETRIEBSSYSTEME, Heinrich-Heine-Universität: L.: *D3OS*. <https://www.cs.hhu.de/en/research-groups/operating-systems/research/translate-to-english-hermes-1>. Version: 2024
- [BLW⁺24] BILLOIR, Eddie ; LABORDE, Romain ; WAZAN, Ahmad S. ; RÜTSCHLÉ, Yves ; BENZEKRI, Abdelmalek: Implementing the principle of least administrative privilege on operating systems: challenges and perspectives. In: *Annals of Telecommunications* 79 (2024), Nr. 11, S. 857–880
- [ECC⁺99] EICKEN, Thorsten von ; CHANG, Chi-Chao ; CZAJKOWSKI, Grzegorz ; HAWBLITZEL, Chris ; HU, Deyu ; SPOONHOWER, Dan: J-Kernel: A Capability-Based Operating System for Java. Version: 1999. <http://dx.doi.org/10.1007/3-540-48749-217>. In: VITEK, Jan (Hrsg.) ; JENSEN, Christian D. (Hrsg.): *Secure Internet Programming: Security Issues for Mobile and Distributed Objects*. Berlin, Heidelberg : Springer Berlin Heidelberg, 1999. – DOI 10.1007/3-540-48749-217. – ISBN 978-3-540-48749-4, 369–393
- [FCK⁺95] FERRAILOLO, David ; CUGINI, Janet ; KUHN, D R. u. a.: Role-based access control (RBAC): Features and motivations. In: *Proceedings of 11th annual computer security application conference*, 1995, S. 241–48
- [Gli79] GLIGOR, V.D.: Review and Revocation of Access Privileges Distributed Through Capabilities. In: *IEEE Transactions on Software Engineering* SE-5 (1979), Nr. 6, S. 575–586. <http://dx.doi.org/10.1109/TSE.1979.230193>. – DOI 10.1109/TSE.1979.230193
- [Goo25] GOOGLE: *Fuchsia – a modern, open source operating system*. <https://fuchsia.dev>, Feb 2025. – Official website; last updated February 28, 2025
- [Han73] HANSEN, Per B.: *Operating system principles*. USA : Prentice-Hall, Inc., 1973. – 232–234 S. – ISBN 0136378439
- [Har88] HARDY, Norm: The Confused Deputy: (or why capabilities might have been invented). In: *ACM SIGOPS Operating Systems Review* 22 (1988), Nr. 4, S. 36–38

- [HCC⁺97] HAWBLITZEL, Chris ; CHANG, Chi-Chao ; CZAJKOWSKI, Grzegorz ; HU, Deyu ; EICKEN, Thorsten von: SLK: A capability system based on safe language technology. In: *Internal Memo* (1997)
- [HE16] HEISER, Gernot ; ELPHINSTONE, Kevin: L4 Microkernels: The Lessons from 20 Years of Research and Deployment. In: *ACM Trans. Comput. Syst.* 34 (2016), April, Nr. 1. <http://dx.doi.org/10.1145/2893177>. – DOI 10.1145/2893177. – ISSN 0734–2071
- [Hei20] HEISER, Gernot: The sel4 microkernel—an introduction. In: *The seL4 Foundation* 1 (2020)
- [HEV⁺98] HEISER, Gernot ; ELPHINSTONE, Kevin ; VOCHTELOO, Jerry ; RUSSELL, Stephen ; LIEDTKE, Jochen: The mungi single-address-space operating system. In: *Software: Practice and Experience* 28 (1998), Nr. 9, 901–928. [http://dx.doi.org/https://doi.org/10.1002/\(SICI\)1097-024X\(19980725\)28:9<901::AID-SPE181>3.0.CO;2-7](http://dx.doi.org/https://doi.org/10.1002/(SICI)1097-024X(19980725)28:9<901::AID-SPE181>3.0.CO;2-7). – DOI [https://doi.org/10.1002/\(SICI\)1097-024X\(19980725\)28:9<901::AID-SPE181>3.0.CO;2-7](https://doi.org/10.1002/(SICI)1097-024X(19980725)28:9<901::AID-SPE181>3.0.CO;2-7)
- [HRJMS13] HERNÁNDEZ-RAMOS, José L ; JARA, Antonio J. ; MARIN, Leandro ; SKARMENTA, Antonio F.: Distributed capability-based access control for the internet of things. In: *Journal of Internet Services and Information Security (JISIS)* 3 (2013), Nr. 3/4, S. 1–16
- [HRK91] HENSKENS, Frans A. ; ROSENBERG, John ; KEEDY, J L.: A capability-based distributed shared memory. (1991)
- [IOA⁺21a] ISAAC, Odun-Ayo ; OKOKPUJIE, Kennedy ; AKINWUMI, Hannah ; JUWE, Jesse ; OTUNUYA, Henry ; ALAGBE, Oladapo: An Overview of Microkernel Based Operating Systems. In: *IOP Conference Series: Materials Science and Engineering* 1107 (2021), apr, Nr. 1, 012052. <http://dx.doi.org/10.1088/1757-899X/1107/1/012052>. – DOI 10.1088/1757–899X/1107/1/012052
- [IOA⁺21b] ISAAC, Odun-Ayo ; OKOKPUJIE, Kennedy ; AKINWUMI, Hannah ; JUWE, Jesse ; OTUNUYA, Henry ; ALAGBE, Oladapo: An overview of microkernel based operating systems. In: *IOP Conference Series: Materials Science and Engineering* Bd. 1107 IOP Publishing, 2021, S. 012052
- [Ka01] KAVAS, A. ; AL. et: Comparing Windows NT, Linux, and QNX as the Basis for Distributed Systems. In: *Technical Report, The Hebrew University of Jerusalem* (2001). – Discusses impersonation on Windows NT and setuid behavior on UNIX-like systems in a security context
- [KAE⁺14] KLEIN, Gerwin ; ANDRONICK, June ; ELPHINSTONE, Kevin ; MURRAY, Toby ; SEWELL, Thomas ; KOLANSKI, Rafal ; HEISER, Gernot: Comprehensive formal verification of an OS microkernel. In: *ACM Trans. Comput. Syst.* 32 (2014), Februar, Nr. 1. <http://dx.doi.org/10.1145/2560537>. – DOI 10.1145/2560537. – ISSN 0734–2071

- [Kar25] KARP, Alan H.: Use Cases for Access Management. (2025), August. <https://www.alanhkarp.com/UseCases.pdf>. – Accessed: 2026-03-23
- [KKLW10] KUZ, Ihor ; KLEIN, Gerwin ; LEWIS, Corey ; WALKER, Adam: capDL: A language for describing capability-based systems. In: *Proceedings of the first ACM asia-pacific workshop on Workshop on systems*, 2010, S. 31–36
- [Lie95] LIEDTKE, Jochen: On micro-kernel construction. In: *ACM SIGOPS Operating Systems Review* 29 (1995), Nr. 5, S. 237–250
- [Lie96] LIEDTKE, Jochen: L4 Reference Manual. (1996), January, Nr. GMD-1021. – Defines the L4 microkernel interface for 32-bit x86 (486/Pentium)
- [Lop18] LOPRIORE, Lanfranco: Password-based protection of clustered segments in distributed memory systems. In: *Journal of Parallel and Distributed Computing* 115 (2018), 29-40. <http://dx.doi.org/https://doi.org/10.1016/j.jpdc.2018.01.003>. – DOI <https://doi.org/10.1016/j.jpdc.2018.01.003>. – ISSN 0743–7315
- [Ope25a] OPENAI: *ChatGPT: Large Language Model*. <https://chat.openai.com>. Version: 2025. – Accessed: 2025-03-11
- [Ope25b] OPENAI: *GitHub Copilot (GPT-4.1)*. <https://github.com/features/copilot>. Version: 2025. – AI-gestütztes Codevervollständigungs-Tool integriert in Rust Rover. Verwendet zur Unterstützung bei der Entwicklung in Rust.
- [Sch26a] SCHNOCK, Julian L.: *D3OS-Capabilities*. <https://git.hhu.de/bsinfo/thesis/ma-jusch322>, 2026
- [Sch26b] SCHNOCK, Julian L.: *Entwicklung einer Capability-basierten Zugriffskontrolle in einem Rust-Betriebssystem*. <https://github.com/Schn0ck/D3OS-Capabilities>, 2026
- [seL25] SEL4 FOUNDATION: *seL4 Documentation*. <https://docs.sel4.systems>, 2025. – Zugriff am 26. Mai 2025
- [SK10] STEINBERG, Udo ; KAUER, Bernhard: NOVA: A Microhypervisor-Based Secure Virtualization Architecture. In: *Proceedings of the 5th European Conference on Computer Systems (EuroSys '10)*. New York, NY, USA : ACM, 2010 (EuroSys), S. 209–222. – Implements a minimal-TCB capability-based microhypervisor
- [SS75] SALTZER, Jerome H. ; SCHROEDER, Michael D.: The protection of information in computer systems. In: *Proceedings of the IEEE* 63 (1975), Nr. 9, S. 1278–1308
- [SSF99] SHAPIRO, Jonathan S. ; SMITH, Jonathan M. ; FARBER, David J.: EROS: a fast capability system. In: *Proceedings of the Seventeenth ACM Symposium on Operating Systems Principles*. New York, NY, USA : Association for Computing Machinery, 1999 (SOSP '99). – ISBN 1581131402, 170–185

- [Sta12] STALLINGS, William: Operating system security. In: *Computer Security Handbook* (2012), S. 24–1
- [Ste13] STEINBERG, Udo: *The NOVA Microhypervisor*. <https://hypervisor.org/nova-fosdem-2013.pdf>, 2013. – Presentation at FOSDEM 2013
- [TB15] TANENBAUM, Andrew S. ; BOS, Herbert: The Classical Thread Model. In: *Modern Operating Systems*. 4. Pearson, 2015. – ISBN 978–0–13–359162–0, Kapitel 2.2.2, S. 98–102
- [WALK10] WATSON, Robert N. ; ANDERSON, Jonathan ; LAURIE, Ben ; KENNAWAY, Kris: Capsicum: Practical Capabilities for {UNIX}. (2010)
- [Wik25a] WIKIPEDIA: *L4 (Mikrokernell)* — *Wikipedia, die freie Enzyklopädie*. [https://de.wikipedia.org/w/index.php?title=L4_\(Mikrokernell\)&oldid=256292324](https://de.wikipedia.org/w/index.php?title=L4_(Mikrokernell)&oldid=256292324). Version: 2025. – [Online; Stand 17. Juni 2025]
- [Wik25b] WIKIPEDIA: *Mikrokernell* — *Wikipedia, die freie Enzyklopädie*. <https://de.wikipedia.org/w/index.php?title=Mikrokernell&oldid=256838379>. Version: 2025. – [Online; Stand 30. Januar 2026]
- [Wik25c] WIKIPEDIA CONTRIBUTORS: *Confused deputy problem* — *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Confused_deputy_problem&oldid=1266870552. Version: 2025. – [Online; accessed 8-April-2025]
- [WWC⁺14] WOODRUFF, Jonathan ; WATSON, Robert N. ; CHISNALL, David ; MOORE, Simon W. ; ANDERSON, Jonathan ; DAVIS, Brooks ; LAURIE, Ben ; NEUMANN, Peter G. ; NORTON, Robert ; ROE, Michael: The CHERI capability model: Revisiting RISC in an age of risk. In: *ACM SIGARCH Computer Architecture News* 42 (2014), Nr. 3, S. 457–468
- [YT05] YUAN, Eric ; TONG, Jin: Attributed based access control (ABAC) for web services. In: *IEEE International Conference on Web Services (ICWS'05)* IEEE, 2005, S. pp. 569